KĨ THUẬT BINARY LIFTING

1. PHẦN MỞ ĐẦU

1.1 Lí do chọn đề tài

Kĩ thuật *Binary Lifting* (hay còn gọi là *Binary Jumping*) đã trở thành một trong những kĩ thuật cơ bản và hiệu quả trong việc giải quyết các bài toán về cây, đặc biệt là trong các bài toán tìm tổ tiên chung gần nhất (LCA) và các vấn đề liên quan đến truy vấn cây. Việc lựa chọn đề tài này xuất phát từ nhu cầu thực tế trong việc tối ưu hóa các thuật toán xử lý cây trong các ứng dụng như tìm kiếm thông tin, quản lý hệ thống, và các bài toán về lý thuyết đồ thị. Tôi lựa chọn đề tài này bởi vì:

- *Binary Lifting* có thể giúp giảm đáng kể thời gian truy vấn LCA từ $O(N)$ xuống $O(logN)$, mang lại sự tối ưu cho các bài toán có cây tĩnh với nhiều truy vấn.
- Kĩ thuật này không chỉ được áp dụng trong các bài toán lý thuyết mà còn được sử dụng trong các hệ thống thực tế như hệ thống cơ sở dữ liệu, phân tích mạng xã hội, tìm kiếm thông tin, và các bài toán tối ưu hóa trong lĩnh vực công nghệ.
- *Binary Lifting* không chỉ giới hạn trong bài toán LCA mà còn có thể được áp dụng trong các bài toán khác liên quan đến cấu trúc cây, giúp tối ưu hóa nhiều quy trình và cải thiện hiệu suất.

Những lý do trên cho thấy việc nghiên cứu và áp dụng *Binary Lifting* là một chủ đề không chỉ có tính học thuật cao mà còn có giá trị thực tiễn lớn, đặc biệt trong bối cảnh các bài toán xử lý dữ liệu lớn đang trở nên ngày càng quan trọng.

1.2 Mục đích của đề tài

Mục đích của đề tài là tìm hiểu, phân tích và áp dụng kĩ thuật *Binary Lifting* để giải quyết các bài toán về tổ tiên chung gần nhất (LCA) trong cây. Cụ thể, đề tài hướng đến những mục tiêu sau:

- Mục tiêu đầu tiên của đề tài là nghiên cứu chi tiết về cách thức hoạt động của kĩ thuật *Binary Lifting*, bao gồm việc xây dựng bảng tổ tiên (Ancestor Table), cách thức truy vấn LCA và tối ưu hóa việc đi lên cây thông qua việc sử dụng các phép toán nhị phân.
- Mục tiêu quan trọng của đề tài là áp dụng kĩ thuật *Binary Lifting* vào việc giải quyết bài toán LCA, từ đó giúp giảm thiểu độ phức tạp thời gian từ $O(N)$ xuống $O(logN)$, mang lại hiệu quả cao trong việc xử lý các cây lớn với nhiều truy vấn.
- Tạo ra bộ bài tập đa dạng, ứng dụng kĩ thuật *Binary Lifting* để học sinh có thể thực hành, từ đó nâng cao khả năng giải quyết bài toán trong các kỳ thi học sinh giỏi. Các bài tập sẽ giúp học sinh hiểu rõ hơn về cách hoạt động của *Binary Lifting* và cách tối ưu hóa thời gian xử lý trong các bài toán cây.

- Cung cấp tài liệu chi tiết về lý thuyết và ứng dụng của kĩ thuật *Binary Lifting*, từ đó hỗ trợ giáo viên trong việc giảng dạy và hướng dẫn học sinh. Tài liệu cũng sẽ bao gồm hướng dẫn giải chi tiết các bài tập và bài toán liên quan.
- Thông qua việc áp dụng *Binary Lifting* trong các bài toán thực tế, học sinh có thể cải thiện khả năng tư duy logic và kỹ năng giải quyết vấn đề, chuẩn bị tốt cho các kỳ thi học sinh giỏi và các bài kiểm tra nâng cao.

Tổng quan, mục đích của đề tài là không chỉ giải quyết vấn đề LCA trong lý thuyết mà còn đưa ra các giải pháp tối ưu cho việc triển khai trong các ứng dụng thực tế, góp phần nâng cao hiệu quả công việc trong các lĩnh vực công nghệ và khoa học máy tính.

2. PHẦN NỘI DUNG

2.1 LÝ THUYẾT

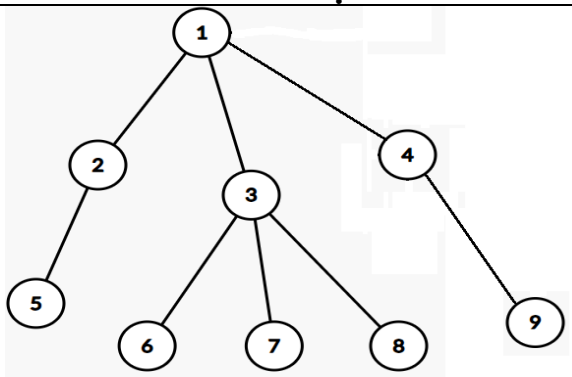
2.1.1 Kĩ thuật Bianry Lifting (Nâng nhị phân)

a) Bài toán

Cho cây G gồm n nút. Cho q truy vấn, mỗi truy vấn gồm 2 số nguyên k và u thể hiện yêu cầu: Tìm tổ tiên thứ k của u trên cây G.

Yêu cầu in ra kết quả của mỗi truy vấn.

- **Dữ liệu vào**: Cho trong file **ANCESTOR.INP**
  - Dòng đầu chứa số nguyên n là số nút của cây
  - $n-1$ dòng tiếp theo, mỗi dòng gồm 2 số nguyên u, v là cạnh của cây
  - Dòng thứ $n+1$ chứa số nguyên q là số lượng truy vấn.
  - q dòng tiếp theo mỗi dòng chứa 2 số nguyên k, u.
- **Kết quả ra**: In kết quả vào file **ANCESTOR.OUT**
  - Mỗi dòng ghi kết quả của 1 truy vấn tương ứng.
- **Ví dụ**

ANCESTOR.INP	ANCESTOR.OUT	Minh họa
9 1 2 1 3 1 4 2 5 3 6 3 7 3 8 4 9 3 1 5 2 6 5 8	2 1 1	 Tổ tiên thứ 1 của nút 5 là 2 Tổ tiên thứ 2 của nút 6 là 1 Tổ tiên thứ 2 của nút 8 là 1

- Giới hạn: $n, q \leq 10^6$

b) Thuật toán

***) Thuật toán duyệt**

Ta dùng mảng $par[u]$ để lưu cha trực tiếp của u. Để tìm tổ tiên thứ k của u ta lặp lại k câu lệnh $u = par[u]$. Nếu u là nút gốc của cây thì $par[u] = -1$

```
int par[N];
int ancestor_k(int u, int k) {
    while (k >= 1 && par[u]!=-1) //Nếu k lớn hơn độ cao của nút
 //u thì dừng không đi lên nữa
    {
        u = par[u];
        --k;
    }
    return u;
}
```

Với thuật toán này ta thấy:

- Độ phức tạp tiền xử lý để tạo mảng là $O(N)$
- Độ phức tạp truy vấn: $O(K) = O(N)$
- Có Q truy vấn nên tổng độ phức tạp là $O(N + Q \times N)$

***) Thuật toán cải tiến**

Để giảm độ phức tạp ta tìm cách tối ưu thời gian truy vấn. Vì vậy nếu thay vì nhảy lên nút cha trực tiếp của 1 nút với độ dài 1 thì ta nhảy lên với độ dài 2 thì thời gian truy vấn giảm xuống còn $O(\frac{K}{2})$. Ta sử dụng thêm mảng $up2[u]$ để lưu trữ tổ tiên thứ 2 của đỉnh u

```
int par[N], up2[N];
void pre() {
    for (int u = 1; u <= n; ++u)
        up2[u] = par[par[u]];
}
int ancestor_k(int u, int k) {
    while (k >= 2 && par[u]!=-1) {
        u = up2[u];      k -= 2;
    }
    if (k >= 1 && par[u]!=-1) {
        u = par[u];      --k;
    }
    return u;
}
```

Để tiếp tục giảm độ phức tạp của thuật toán ta có thể nhảy lên với bước dài hơn với độ dài 4 thì thời gian truy vấn giảm xuống còn $O(\frac{K}{4})$. Ta cần dùng thêm mảng $up4[]$

```
int par[N], up2[N], up4[N];
void pre()
{
     //tính tổ tiên thứ 2 của u
    for (int u = 1; u <= n; ++u) up2[u] = par[par[u]];
     //tính tổ tiên thứ 4 của u
    for (int u = 1; u <= n; ++u) up4[u] = up2[up2[u]];
}
int ancestor_k(int u, int k)
```

```
{
    while(k>=4&&par[u]!=-1) //di chuyển lên tổ tiên thứ 4 của u
    {  u = up4[u];    k -= 4;   }
    if (k >= 2 && par[u]!=-1)//di chuyển lên tổ tiên thứ 2 của u
    {  u = up2[u];    k -= 2;   }
    if (k>=1&&par[u]!=-1)//Di chuyển lên tổ tiên trực tiếp của u
    {  u = par[u];    --k;    }
    return u;
}
```

Tiếp đó ta có thể nhảy lên với độ dài $8, 16, 32$ ... để giảm độ phức tạp của thuật toán thì ta sẽ cần dùng log_2N mảng up. Khi đó độ phức tạp là:

- Tiền xử lý: $O(N \times log_2N)$
- Truy vấn: $O(log_2N)$
- Tổng độ phức tạp: $O(N \times log_2N + Q \times log_2N)$
- Độ phức tạp bộ nhớ: $O(N \times log_2N)$ (mảng par và log_2N mảng up)

***) Tiền xử lý trong kĩ thuật binary lifting**

Với cách cải tiến thuật toán như trên ta cần dùng log_2N mảng up. Điều này sẽ làm cho chương trình cồng kềnh, không khoa học và khó sửa chữa nên ta tạo một mảng $up[][]$ với ý nghĩa up[i][j] lưu cha thứ 2^j của nút i (tương ứng mảng $up2^j[u]$ như mô tả thuật toán cải tiến phía trên).

+ Ban đầu khởi tạo tất cả các phần tử của mảng bằng -1

+ Thực hiện DFS tính up[i][0] cho tất cả các nút i (tìm nút cha trực tiếp của nút i)

+ Vì $2^j = 2^{j-1} + 2^{j-1}$ nên để tìm cha thứ 2^j của nút i thì ta phải thực hiện 2 bước nhảy có kích thước 2^{j-1}. Sau bước nhảy đầu tiên từ nút i ta sẽ tới được nút cha thứ 2^{j-1} của i ($up[i][j-1]$), sau lần nhảy thứ 2 thì từ nút vừa đến ta lại đi lên được 2^{j-1} nút cha nữa, tức là đến $up[[i][j-1]][j-1]$. Vì vậy ta có công thức để tìm nút cha thứ j ($j > 0$) của nút i:

$$up[i][j] = up[up[i][j-1]][j-1], khi\ up[i][j-1] != -1$$

Đây là cách tiền xử lý của kĩ thuật *Binary Lifting*

***) Truy vấn trong binary lifting:**

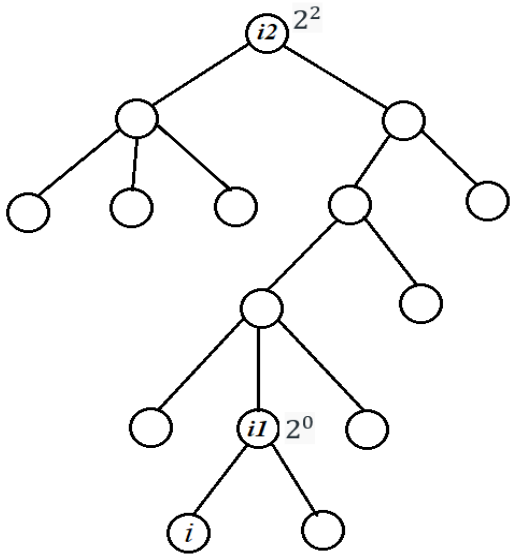
Với mỗi truy vấn tìm nút cha thứ k của nút i ta cần biểu diễn nhị phân số nguyên k. Nếu bit thứ j của k là 1 thì ta nhảy lên tổ tiên thứ 2^j của nó.

Số nguyên k luôn biểu diễn được thành tổng lũy thừa của 2 như sau:

$$k = 2^{p1} + 2^{p2} + \cdots + 2^{pm}\ với\ 0 \le p_m < p_{m-1} < \cdots < p_2 < p_1 < log_2k$$

Vậy từ nút i ta nhảy lên nút i_1 là cha thứ 2^{pm} của i, từ nút i_1 ta nhảy lên nút i_2 là cha thứ $2^{p_{m-1}}$ của i_2, … cứ như vậy lên đến nút cha thứ k của i.

Ví dụ ta cần tìm nút cha thứ 5 của nút i; khi đó $k = 5 = 2^2 + 2^0$. Khi đó từ nút i ta nhảy 2^0 bước lên nút $i1$ và tiếp tục từ $i1$ nhảy 2^2 bước lên $i2$. Nút $i2$ chính là nút cha thứ 5 của nút i



Thêm ví dụ với $k = 100$:

Nếu chúng ta cần tìm nút cha thứ 100 của nút X nhưng trong bảng up chỉ có các nút cha thứ $1, 2, 4, 8, 16, 32, 64, \ldots$ của nút hiện tại. Vì vậy, ta sẽ cần nhảy tới nút cha lớn nhất có thể của X mà không vượt quá 100.

Ta thấy $100 = 64 + 32 + 4 = 2^6 + 2^5 + 2^2 = (1100100)_2$

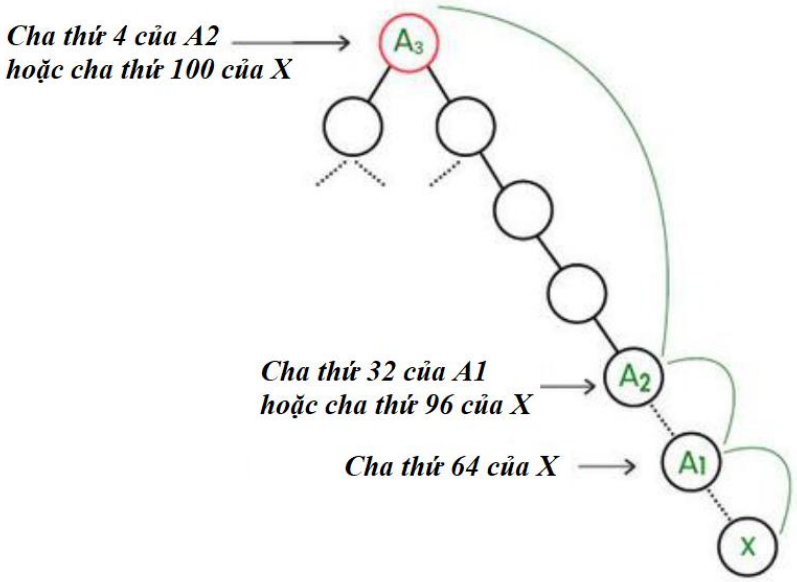
Như vậy:

+ Nhảy bước đầu tiên đến nút $A1$ là cha thứ 64 của X: $up[X][6] = A1$

+ Nhảy bước thứ 2 đến nút $A2$ là cha thứ 32 của $A1$: $up[A1][5] = A2$

+ Nhảy bước thứ 3 đến nút $A3$ là cha thứ 4 của nút $A2$: $up[A2][2] = A3$

và cũng là cha thứ 100 của nút X. Các thao tác nhảy như hình sau:



Kết luận: *Binary lifting* là một phương pháp quy hoạch động trên cây, trong kĩ thuật này ta cần tính toán trước một số tổ tiên của mỗi nút để trả lời một số lượng lớn các truy vấn tìm tổ tiên của một nút bất kì trong thời gian logarit. Về tính chất, kĩ thuật này khá

giống với chặt nhị phân, chỉ khác mỗi lần nhảy thì *binary lifting* nhảy 2^k đơn vị còn chặt nhị phân nhảy $\frac{l-r+1}{2}$ đơn vị.

c) Chương trình

```
#include <bits/stdc++.h>
using namespace std;
vector<vector<int>> up(n + 1, vector<int>(LOG, -1));
 vector<char> vis(n + 1, 0);
 queue<int> q;
vector<vector<int>> g(n + 1);
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    freopen("ancestor.inp","r",stdin);
    freopen("ancestor.out","w",stdout);
    int n;
    cin >> n;
    int LOG = 1;
    while ((1 << LOG) <= n) LOG++;
    for (int i = 1; i <= n - 1; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    int root = 1;
    vis[root] = 1;
    up[root][0] = -1;
    q.push(root);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) if (!vis[v]) {
            vis[v] = 1;
            up[v][0] = u;
            q.push(v);
        }
    }
    for (int j = 1; j < LOG; j++) {
        for (int u = 1; u <= n; u++) {
            int mid = up[u][j - 1];
            up[u][j] = (mid == -1 ? -1 : up[mid][j - 1]);
```

```
        }
    }
    int Q;
    cin >> Q;
    while (Q--) {
        int k, u;
        cin >> k >> u;
        for (int j = LOG - 1; j >= 0; j--) {
            if (u == -1) break;
            if (k & (1 << j)) u = up[u][j];
        }
        cout << u << "\n";
    }
    return 0;
}
```

d) Độ phức tạp

Độ phức tạp tiền xử lý: $O(NlogN)$

Độ phức tạp truy vấn: $O(QlogN)$

Tổng độ phức tạp thời gian: $O(Nlog(N) + Qlog(N))$

Độ phức tạp không gian: $O(NlogN)$

2.1.2 Các trường hợp sử dụng Binary Lifting

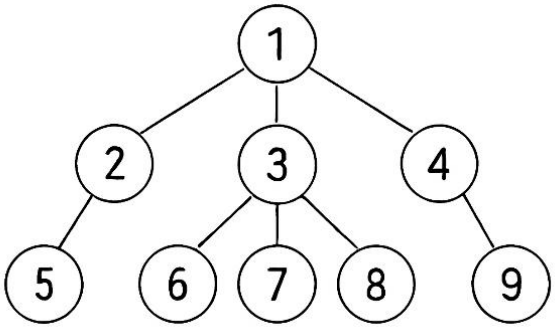
2.1.2.1 Tìm tổ tiên bất kì của 1 nút

Binary Lifting được sử dụng để trả lời một số lượng lớn các truy vấn, trong đó mỗi truy vấn yêu cầu tính toán một tổ tiên tùy ý của một nút bất kỳ. Bài toán này đã được trình bày chi tiết trong phần 2.1.1.

2.1.2.2 Tìm tổ tiên chung gần nhất của 2 nút trên cây.

a) <u>Khái niệm tổ tiên chung gần nhất</u>

Giả sử G là một cây. Tổ tiên chung gần nhất (Lowest Common Ancestor – LCA) của 2 nút u và v là nút w trong cây, w là tổ tiên của cả u và v và cách xa gốc nhất. Nếu u là tổ tiên của v thì $LCA(u, v) = u$. Ngược lại, nếu v là tổ tiên của u thì $LCA(u, v) = v$.



Như hình vẽ trên ta thấy có: $LCA(6,5)$ là 1. $LCA(7,1)$ là 1. $LCA(6,8)$ là 3.

b) <u>Bài toán:</u>

Cho cây G và q truy vấn, mỗi truy vấn là tìm $LCA(u, v)$. Yêu cầu in ra kết quả của mỗi truy vấn.

- **Dữ liệu vào**: Cho trong file **LCA.INP**
  - Dòng đầu chứa số nguyên n, m là số nút của cây và số cạnh trên cây
  - m dòng tiếp theo, mỗi dòng gồm 2 số nguyên u, v là cạnh của cây
  - Dòng thứ $m + 2$ chứa số nguyên q là số lượng truy vấn
  - q dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v.
- **Kết quả ra**: In kết quả vào file **LCA.OUT**
  - Mỗi dòng ghi kết quả của 1 truy vấn tương ứng.

- **Ví dụ**

LCA.INP	LCA.OUT
9 1 2 1 3 1 4 2 5 3 6 3 7 3 8 4 9 3 6 5 7 1 6 8	1 1 3

- Giới hạn: $n, m, q \leq 10^6$

<u>c) Thuật toán</u>

Binary Lifting là một kĩ thuật quy hoạch động giúp tìm tổ tiên chung gần nhất (LCA) của hai nút trong cây một cách hiệu quả. Cách làm này sử dụng một bảng (up) để lưu trữ

các tổ tiên bậc 2^j của các nút. Bảng này được tính trước, cho phép chúng ta truy vấn các tổ tiên bậc cao một cách nhanh chóng.

Các bước chính trong Binary Lifting:

- Khởi tạo mảng up: $up[i][j]$ lưu trữ tổ tiên 2^jcủa nút i.

Để tính $up[i][j]$, chúng ta phải tìm tổ tiên 2^{j-1}của nút i và sau đó tìm tổ tiên 2^{j-1} của tổ tiên đó.

Công thức: $up[i][j] = up[up[i][j-1]][j-1]$

Khởi tạo tổ tiên bậc 0:

Tổ tiên bậc 2^0 (tức là cha trực tiếp) của mỗi nút được tính toán:

$$up[i][0] = parent[i]$$

Cập nhật bảng up: Sau khi đã tính toán tổ tiên bậc 2^0, các tổ tiên bậc cao hơn được tính bằng cách sử dụng công thức trên.

Tìm LCA của hai nút:

- Nếu một nút là tổ tiên của nút kia, thì nút đó chính là LCA.
- Nếu hai nút không ở cùng một cấp độ, nâng nút thấp hơn lên cùng cấp độ với nút còn lại.
- Tìm LCA: Sau khi nâng chúng lên cùng cấp độ, kiểm tra tổ tiên của cả hai nút, nếu không bằng nhau thì tiếp tục nâng chúng lên cho đến khi gặp nhau.

c) <u>Chương trình</u>

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1000005;
const int MAX_LOG = 20;
vector<int> g[MAXN];
int up[MAXN][MAX_LOG];
int depth[MAXN];
int n, m;
void build(int root) {
    queue<int> q;
    q.push(root);
    depth[root] = 0;
    up[root][0] = -1;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : g[u]) {
```

```
            if (v == up[u][0]) continue;

            depth[v] = depth[u] + 1;
            up[v][0] = u;

            for (int j = 1; j < MAX_LOG; j++) {
                if (up[v][j - 1] != -1)
                    up[v][j] = up[ up[v][j - 1] ][j - 1];
                else
                    up[v][j] = -1;
            }

            q.push(v);
        }
    }
}
int lca(int u, int v) {
    if (depth[u] < depth[v])
        swap(u, v);
    for (int i = MAX_LOG - 1; i >= 0; i--) {
        if (up[u][i] != -1 && depth[ up[u][i] ] >= depth[v])
            u = up[u][i];
    }
    if (u == v) return u;
    for (int i = MAX_LOG - 1; i >= 0; i--) {
        if (up[u][i] != -1 && up[u][i] != up[v][i]) {
            u = up[u][i];
            v = up[v][i];
        }
    }
    return up[u][0];
}
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
   // freopen("LCA.INP", "r", stdin);
   // freopen("LCA.OUT", "w", stdout);
    cin >> n;
    m = n - 1;   // vì cây có n-1 cạnh
    for (int i = 1; i <= n; i++) {
        g[i].clear();
```

```
            for (int j = 0; j < MAX_LOG; j++)
                up[i][j] = -1;
        }
        for (int i = 1; i <= m; i++) {
            int u, v;
            cin >> u >> v;
            g[u].push_back(v);
            g[v].push_back(u);
        }
        build(1);
        int q;
        cin >> q;
        while (q--) {
            int u, v;
            cin >> u >> v;
            cout << lca(u, v) << '\n';
        }
        return 0;
}
```

d) <u>Độ phức tạp</u>

Tính các tổ tiên mất $O(Nlog\ N)$.

Mỗi truy vấn LCA có độ phức tạp là $O(log\ N)$, vì chúng ta chỉ cần "nhảy" qua các tổ tiên bậc 2^j.

Vậy tổng độ phức tạp thời gian là: $O(Nlog\ N)$.

Độ phức tạp không gian là: $O(Nlog\ N)$.

2.1.2.3 Tìm trọng số lớn nhất trên đường đi giữa 2 đỉnh trên cây có trọng số

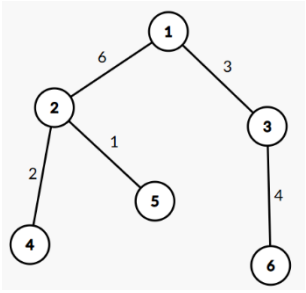
<u>a) Bài toán</u>

Cho cây G gồm n nút và m cạnh. Mỗi cạnh (u, v) của cây có trọng số w. Yêu cầu: Tìm trọng số lớn nhất giữa đường đi từ u đến v trên cây G.

- **Dữ liệu vào:** Cho trong file **TRONGSO.INP**
    - Dòng đầu chứa số nguyên n, m là số nút của cây và số cạnh trên cây
    - m dòng tiếp theo, mỗi dòng gồm 3 số nguyên u, v, w thể hiện cạnh (u, v) có trọng số w.
    - Dòng thứ $m + 2$ chứa số nguyên q là số lượng truy vấn
    - q dòng tiếp theo mỗi dòng chứa 2 số nguyên u, v.
- **Kết quả ra:** In kết quả vào file **TRONGSO.OUT**

- Mỗi dòng ghi kết quả của 1 truy vấn tương ứng.

- **Ví dụ**

TRONGSO.INP	TRONGSO.OUT
7 6 1 2 6 1 3 3 3 4 4 4 5 2 4 6 1 5 7 5 3 1 4 1 7 4 7	4 5 5 

- Giới hạn: $n, m, q \leq 10^6$

<u>b) Thuật toán</u>

Ta có một cây có trọng số và có Q truy vấn. Trong mỗi truy vấn, ta được cho 2 đỉnh (X và Y) và cần tìm trọng số lớn nhất trong tất cả các cạnh nằm trên đường đi từ X đến Y.

Ta lưu thêm thông tin vào bảng $up[][]$. Trước đây, $up[i][j]$ chỉ lưu tổ tiên thứ 2^j của đỉnh i, thì bây giờ, ta sẽ lưu thêm trọng số lớn nhất của các cạnh nằm trên đường đi khi ta nhảy từ đỉnh i đến tổ tiên thứ 2^j của nó.

$up[i][j].first$ là tổ tiên thứ 2^j của nút i.

$up[i][j].second$ là trọng số cạnh lớn nhất trong đường đi từ i đến tổ tiên thứ 2^j

Để tính kết quả cho truy vấn, ta sẽ tìm LCA của X và Y bằng cách nâng chúng lên cùng 1 level rồi nâng dần tổ tiên của cả 2 nút cho đến khi chúng gặp nhau ở $LCA(X, Y)$. Khi nâng X và Y lên ta cũng theo dõi trọng số cạnh lớn nhất mà ta đã đi qua.

Để tìm trọng số lớn nhất từ X đến Y ta chia đường đi từ X đến Y thành 2 phần:

- Đường đi từ X đến $LCA(X, Y)$
- Đường đi từ Y đến $LCA(X, Y)$

Cuối cùng, ta trả về trọng số lớn nhất trong cả hai đoạn đường đó vì $LCA(X, Y)$ chia đoạn đường đi từ X đến Y thành 2 phần không giao nhau (mỗi phần thuộc 1 nhánh của cây). Ví dụ ta có cây sau:

Tìm trọng số cạnh lớn nhất giữa nút 4 và nút 6:

LCA(4, 6) = 1 (tổ tiên chung gần nhất của 4 và 6).

Đoạn từ 4 đến LCA(4, 6) là: 4→2→1, trọng số cạnh lớn nhất là 6.

Đoạn từ 6 đến LCA(4, 6) là: 6→3→1, trọng số cạnh lớn nhất là 4.

Kết quả là max(6, 4) = 6, nên trọng số cạnh lớn nhất giữa 4 và 6 là 6.

c) Chương trình

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1000005;
const int MAXLOG = 20;
vector<pair<int,int>> g[MAXN];
int up[MAXN][MAXLOG];
int maxW[MAXN][MAXLOG];
int depth[MAXN];
int n, m;
void build(int root) {
    queue<int> q;
    q.push(root);
    depth[root] = 0;
    up[root][0] = -1;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto edge : g[u]) {
            int v = edge.first;
            int w = edge.second;
            if (v == up[u][0]) continue;
            depth[v] = depth[u] + 1;
            up[v][0] = u;
            maxW[v][0] = w;
            for (int j = 1; j < MAXLOG; j++) {
                if (up[v][j-1] != -1) {
                    up[v][j] = up[ up[v][j-1] ][j-1];
                    maxW[v][j] = max(maxW[v][j-1],maxW[ up[v][j-1] ][j-1]);
                } else {
                    up[v][j] = -1;
                }
            }

            q.push(v);
        }
    }
}
int query(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int ans = 0;
    for (int i = MAXLOG-1; i >= 0; i--) {
```

```
        if (up[u][i] != -1 && depth[ up[u][i] ] >= depth[v]) {
            ans = max(ans, maxW[u][i]);
            u = up[u][i];
        }
    }

    if (u == v) return ans;
    for (int i = MAXLOG-1; i >= 0; i--) {
        if (up[u][i] != -1 && up[u][i] != up[v][i]) {
            ans = max(ans, maxW[u][i]);
            ans = max(ans, maxW[v][i]);
            u = up[u][i];
            v = up[v][i];
        }
    }
    ans = max(ans, maxW[u][0]);
    ans = max(ans, maxW[v][0]);
    return ans;
}
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    memset(up, -1, sizeof(up));
    memset(maxW, 0, sizeof(maxW));
    build(1);
    int q;
    cin >> q;
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << query(u, v) << "\n";
    }
    return 0;
}
```

d) Độ phức tạp

Độ phức tạp thời gian: $O(NlogN + QlogN)$

Độ phức tạp không gian: $O(NlogN)$

2.1.2.4 Tìm tổ tiên thứ k của 1 nút trên cây nhị phân

a) <u>Bài toán</u>

Cho một cây nhị phân trong đó các nút được đánh số từ 1 đến n. Cho một nút m $(1 \leq m \leq n)$ và một số nguyên dương K, bạn cần in ra tổ tiên thứ K của nút đó trong cây nhị phân. Nếu không tồn tại tổ tiên như vậy, in ra -1.

Lưu ý:

- Đảm bảo rằng nút tồn tại trong cây.
- Tất cả các nút của cây có các giá trị khác nhau.

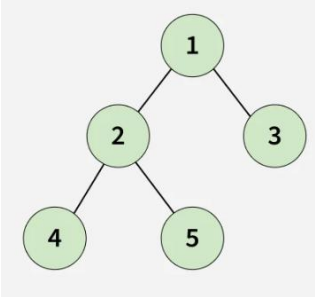
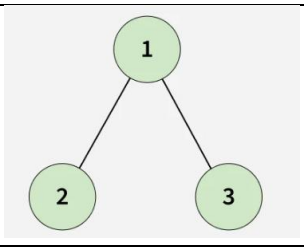
Dữ liệu vào: Cho trong file **KANCESTOR.INP**

- Dòng đầu chứa số n, k và m.

Kết quả ra: In ra file **KANCESTOR.OUT**

- In ra tổ tiên thứ k của nút m

Ví dụ:

KANCESTOR.INP	KANCESTOR.OUT	Giải thích	
5 2 4	1	Tổ tiên thứ 2 của nút 4 và 5 là 1. Tổ tiên thứ 3 của nút 4 sẽ là -1.	
3 1 3	1	Tổ tiên thứ 1 của nút 3 là 1	

Giới hạn: $1 \leq n \leq 10^5$; $1 \leq k \leq 100$;

b) <u>Thuật toán</u>

- Duyệt cây theo DFS để đi qua tất cả các nút trong cây và xây dựng bảng tổ tiên. Tại mỗi nút, lưu tổ tiên của nó ở các khoảng cách 2^i. Tức là, nếu chúng ta có tổ tiên ở mức 2^0, 2^1, 2^2,… thì tổ tiên thứ 2^i của một nút có thể được truy xuất trong bảng tổ tiên.

- Bảng tổ tiên: Dùng mảng $dp[node][i]$ để lưu trữ tổ tiên thứ 2^i của nút $node$. Nếu tổ tiên thứ 2^i tồn tại, chúng ta sẽ cập nhật giá trị trong bảng, nếu không, để giá trị là -1.

- Truy vấn tổ tiên thứ K của một nút, chúng ta phân tích K dưới dạng nhị phân và lần lượt truy xuất các tổ tiên từ bảng dp tương ứng với các bit 1 của K (mỗi bit 1 trong biểu diễn nhị phân của K sẽ tương ứng với một tổ tiên ở mức 2^i).

c) <u>Chương trình</u>

```
#include <bits/stdc++.h>
using namespace std;
vector<vector<int>> up(n + 1, vector<int>(LOG));
int LOG = 20;
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n, k, m;
    cin >> n >> k >> m;

    for (int i = 1; i <= n; i++) {
        if (i == 1)
            up[i][0] = -1;
        else
            up[i][0] = i / 2;
    }
    for (int j = 1; j < LOG; j++) {
        for (int i = 1; i <= n; i++) {
            if (up[i][j - 1] != -1)
                up[i][j] = up[ up[i][j - 1] ][j - 1];
            else
                up[i][j] = -1;
        }
    }
    for (int j = 0; j < LOG; j++) {
        if (k & (1 << j)) {
            m = up[m][j];
            if (m == -1) break;
        }
    }
    cout << m;
}
```

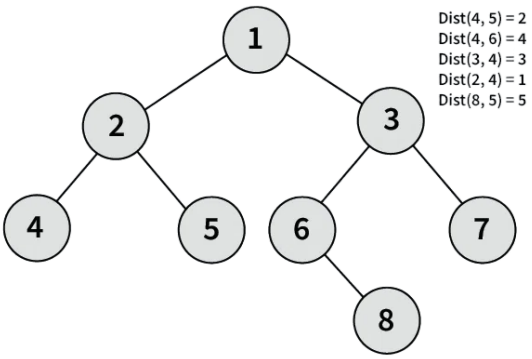
d) <u>Độ phức tạp</u>

- Xây dựng bảng tổ tiên: $O(nlog\ n)$, với n là số nút trong cây và $logn$ là số mức tổ tiên cần lưu.
- Tìm tổ tiên thứ K: $O(log\ n)$, vì mỗi lần truy vấn tổ tiên, chúng ta chỉ cần đi qua $logn$ mức tổ tiên.

2.1.2.5 Tìm khoảng cách giữa 2 nút trên cây nhị phân

a) <u>Bài toán</u>

Cho cây nhị phân, tìm khoảng cách giữa 2 nút trên cây không có con trỏ cha (tức là không thể truy cập ngược từ một nút đến cha của nó). Khoảng cách giữa 2 nút là số cạnh tối thiểu cần phải đi qua để đến một nút từ một nút khác. Hai nút đã cho được đảm bảo nằm trong cây nhị phân và tất cả các giá trị nút là duy nhất.



Dữ liệu vào: Cho trong file **DISTANCE.INP**

- Dòng đầu chứa số nguyên n là số nút của cây
- $n-1$ dòng tiếp theo mỗi dòng chứa 2 số nguyên u và v mô tả 1 cạnh của cây.
- Dòng tiếp theo chứa số nguyên q là số lượng truy vấn trên cây
- q dòng tiếp theo mỗi dòng chứa 2 số nguyên $u\ v$ mô tả 1 truy vấn

Kết quả ra: In ra file **DISTANCE. OUT**

- Mỗi dòng trong q dòng in ra 1 số nguyên là khoảng cách giữa nút u và v trong cây.

Ví dụ:

DISTANCE.INP	DISTANCE. OUT
8 1 2 1 3 2 4 2 5 3 6 3 7 6 8	2 4 3 1 5

5 4 5 4 6 3 4 2 4 8 5	

b) <u>Thuật toán</u>

Sử dụng *Binary lifting* để lưu trữ tổ tiên ở các cấp khác nhau: $up[node][j]$ lưu tổ tiên mức 2^j của node.

Tìm LCA của hai nút u và v bằng cách nhảy qua các tổ tiên nhờ mảng up.

Tìm khoảng cách giữa 2 nút bằng công thức sau:

$$depth[u] + depth[v] - 2 * depth[lca]$$

Trong đó:

- $depth[u]$ và $depth[v]$ là độ sâu của các nút u và v trong cây, tức là số bước (hoặc số cạnh) từ gốc cây đến nút đó.
- $depth[lca]$ là độ sâu của LCA, tức là số bước từ gốc cây đến tổ tiên chung thấp nhất của hai nút u và v.

c) <u>Chương trình</u>

```
#include <bits/stdc++.h>

using namespace std;
const int MAXN = 100005;
const int LOG = 17;
vector<int> adj[MAXN];
int up[MAXN][LOG];
int depth[MAXN];
int qnum;
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
```

```
for (int i = 1; i <= n; i++) {
    depth[i] = -1;
    for (int j = 0; j < LOG; j++)
        up[i][j] = -1;
}
queue<int> q;
q.push(1);
depth[1] = 0;
while (!q.empty()) {
    int u = q.front();
    q.pop();
    for (int v : adj[u]) {
        if (depth[v] == -1) {
            depth[v] = depth[u] + 1;
            up[v][0] = u;

            for (int j = 1; j < LOG; j++) {
                if (up[v][j - 1] != -1)
                    up[v][j] = up[ up[v][j - 1] ][j - 1];
            }
            q.push(v);
        }
    }
}
cin >> qnum;
while (qnum--) {
    int u, v;
    cin >> u >> v;
    int uu = u, vv = v;
    if (depth[uu] < depth[vv])
        swap(uu, vv);
    for (int j = LOG - 1; j >= 0; j--) {
        if (up[uu][j] != -1 &&
            depth[ up[uu][j] ] >= depth[vv]) {
            uu = up[uu][j];
        }
    }
    int lca;
    if (uu == vv) {
        lca = uu;
    } else {
```

```
                for (int j = LOG - 1; j >= 0; j--) {
                    if (up[uu][j] != -1 &&
                        up[uu][j] != up[vv][j]) {
                        uu = up[uu][j];
                        vv = up[vv][j];
                    }
                }
                lca = up[uu][0];
            }
            int dist = depth[u] + depth[v] - 2 * depth[lca];
            cout << dist << "\n";
        }
        return 0;
}
```

d) <u>Độ phức tạp</u>

- Thời gian:
  - Xây dựng bảng tổ tiên: $O(nlogn)$, vì mỗi lần duyệt qua cây, ta xây dựng bảng tổ tiên cho mỗi nút.
  - Mỗi truy vấn LCA: $O(logn)$ nhờ Binary Lifting.

    Mỗi truy vấn khoảng cách: $O(logn)$ (gồm tìm LCA và tính khoảng cách).
- Bộ nhớ: $O(nlogn)$ cho bảng tổ tiên.

2.1.2.6 Xây dựng đường đi tối ưu

a) <u>Bài toán</u>

Cho một đồ thị có kết nối với N nút và $N - 1$ cạnh hai chiều, được cho bởi một mảng 2 chiều $edges[][]$, trong đó $edges[i] = \{u, v, w\}$ có nghĩa là có một cạnh hai chiều giữa nút u và nút v với trọng số w. Nhiệm vụ của bạn là trả lời Q truy vấn sao cho với mỗi truy vấn, $query[i] = \{u, v, w\}$, in ra "1" nếu việc thêm một cạnh mới giữa nút u và v với trọng số w sẽ tạo ra một MST (Cây bao trùm tối thiểu) tốt hơn, nếu không thì in ra "0".

Lưu ý: Mỗi truy vấn độc lập với nhau.

Dữ liệu vào: Cho trong file **OPTIMAL.INP**

- Dòng đầu: chứa số nguyên N
- $N - 1$ dòng tiếp theo, mỗi dòng chứ 3 số nguyên u, v, w thể hiện 1 cạnh của câ
- Dòng tiếp theo chứa số nguyên Q
- Q dòng tiếp theo mỗi dòng chứa 3 số nguyên u, v, w thể hiện 1 cạnh mới

Kết quả ra: In ra file **OPTIMAL.OUT**

- In ra số 1 nếu thêm cạnh mới, in ra 0 nếu không thêm cạnh mới.

Ví dụ:

OPTIMAL.INP	OPTIMAL.OUT	Giải thích
5 1 2 2 1 3 3 3 4 5 3 5 4 2 1 4 2 4 5 8	1 0	- Với truy vấn 1: Nếu xóa cạnh (3,4) rồi thêm {1, 4, 2} thì ta có thể giảm tổng trọng số đi 3 đơn vị - Với truy vấn 2: Việc thêm cạnh {4, 5, 8} không làm giảm chi phí tối thiểu của MST
3 1 2 5 1 3 6 2 2 3 2 2 3 12	1 0	- Với truy vấn 1, nếu xóa cạnh (1, 3) và thêm cạnh {2, 3, 2} thì có thể giảm tổng trọng số đi 4 đơn vị - Với truy vấn 2, thêm cạnh {2, 3, 12} sẽ không làm giảm chi phí tối thiểu của MST

b) <u>Thuật toán</u>

Vì trong mỗi truy vấn, chúng ta được cho một cạnh và cần xác định xem việc thêm cạnh này vào đồ thị có làm giảm chi phí của cây khung nhỏ nhất (MST) hay không, nên việc thêm một cạnh sẽ tạo ra một chu trình.

Để giảm chi phí của MST, cạnh mới này phải có trọng số nhỏ hơn so với cạnh có trọng số lớn nhất trong chu trình. Giả sử chúng ta thêm một cạnh (u, v) với trọng số w và các nút u và v có $LCA(u, v)$ là L, thì để giảm chi phí của MST, w phải nhỏ hơn trọng số của cạnh lớn nhất trong chu trình.

Để tìm trọng số lớn nhất trong chu trình, chúng ta có thể tìm trọng số lớn nhất từ nút L đến nút u và từ nút L đến nút v. Trọng số lớn nhất giữa hai đường đi này sẽ là trọng số lớn nhất trong chu trình. Để tìm LCA và trọng số cạnh lớn nhất một cách hiệu quả cho mỗi truy vấn, ta có thể sử dụng kĩ thuật *Binary Lifting* để lưu trữ các tổ tiên ($LCA[][]$) cũng như trọng số cạnh lớn nhất giữa nút và tổ tiên ($MWE[][]$).

- Xây dựng bảng $LCA[i][j]$ sẽ lưu tổ tiên thứ 2^j của nút i.
- Xây dựng bảng $MWE[i][j]$ sẽ lưu trọng số cạnh lớn nhất từ nút i đến tổ tiên thứ 2^j của nút i. Ta có thể tìm LCA của hai nút và trọng số cạnh lớn nhất trong chu trình chỉ trong thời gian $log(N)$.
- Với mỗi truy vấn, tìm trọng số của cạnh lớn nhất trong đường đi từ u đến $LCA(u, v)$ và từ v đến $LCA(u, v)$, sau đó kiểm tra xem trọng số đó có nhỏ hơn hay lớn hơn trọng số của cạnh mới. Nếu trọng số lớn nhất nhỏ hơn hoặc bằng trọng số của cạnh mới, thì việc thêm cạnh mới sẽ không làm giảm trọng số của

MST. Ngược lại, nếu trọng số lớn nhất lớn hơn trọng số của cạnh mới, việc thêm cạnh mới sẽ làm giảm trọng số của MST.

c) <u>Chương trình</u>

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 100005;
const int LOG = 17;
vector<pair<int,int>> adj[MAXN];
int up[MAXN][LOG];
int maxEdge[MAXN][LOG];
int depth[MAXN];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    freopen("OPTIMAL.INP","r",stdin);
    freopen("OPTIMAL.OUT","w",stdout);
    int N;
    cin >> N;
    for (int i = 1; i <= N; i++)
        adj[i].clear();
    for (int i = 0; i < N - 1; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
    for (int i = 1; i <= N; i++) {
        depth[i] = -1;
        for (int j = 0; j < LOG; j++) {
            up[i][j] = -1;
            maxEdge[i][j] = 0;
        }
    }
    queue<int> q;
    q.push(1);
    depth[1] = 0;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```
        for (auto edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;
            if (depth[v] == -1) {
                depth[v] = depth[u] + 1;
                up[v][0] = u;
                maxEdge[v][0] = w;
                for (int j = 1; j < LOG; j++) {
                    if (up[v][j-1] != -1) {
                        up[v][j] = up[ up[v][j-1] ][j-1];
                        maxEdge[v][j] = max(maxEdge[v][j-1],
                                            maxEdge[ up[v][j-1] ][j-1]);
                    }
                }

                q.push(v);
            }
        }
    }
    int Q;
    cin >> Q;
    while (Q--) {
        int u, v, w;
        cin >> u >> v >> w;
        int uu = u, vv = v;
        int maxW = 0;
        if (depth[uu] < depth[vv])
            swap(uu, vv);
        for (int j = LOG - 1; j >= 0; j--) {
            if (up[uu][j] != -1 &&
                depth[ up[uu][j] ] >= depth[vv]) {
                maxW = max(maxW, maxEdge[uu][j]);
                uu = up[uu][j];
            }
        }
        if (uu != vv) {
            for (int j = LOG - 1; j >= 0; j--) {
                if (up[uu][j] != -1 &&
                    up[uu][j] != up[vv][j]) {
                    maxW = max(maxW, maxEdge[uu][j]);
                    maxW = max(maxW, maxEdge[vv][j]);
```

```
                uu = up[uu][j];
                vv = up[vv][j];
            }
        }
        maxW = max(maxW, maxEdge[uu][0]);
        maxW = max(maxW, maxEdge[vv][0]);
    }
    if (maxW > w)
        cout << 1 << "\n";
    else
        cout << 0 << "\n";
  }
}
```

d) Độ phức tạp

- Độ phức tạp về thời gian: $O(Nlog(N) + Q \log(N))$, trong đó N là số nút và Q là số truy vấn.
- Độ phức tạp không gian: $O(Nlog(N))$, do bảng $LCA[][]$ và $MWE[][]$.

2.1.2.7 Tìm trọng số cạnh trong cây khung với cạnh (u, v)

a) <u>Bài toán</u>

Cho đồ thị vô hướng có trọng số với N đỉnh và M cạnh, nhiệm vụ là đối với mỗi cạnh (u, v), tìm trọng số tối thiểu có thể của Cây bao trùm chứa cạnh (u, v). Các cạnh được cho dưới dạng một mảng 2 chiều edges[][], trong đó $edges[i] = \{u, v, w\}$ có nghĩa là có một cạnh từ đỉnh u đến đỉnh v với trọng số w.

Dữ liệu đầu vào: Cho trong file **WEIGHTS.INP**

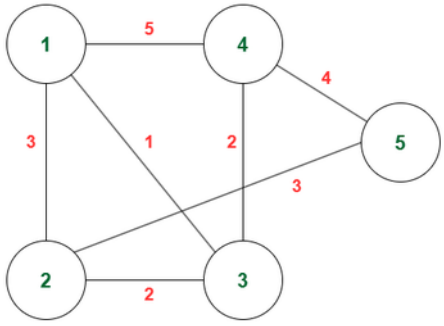
- Dòng đầu chứa 3 số nguyên n, m
- m dòng tiếp theo mỗi dòng chứa 3 số u, v, w mô tả 1 cạnh

Kết quả ra: In ra file **WEIGHTS.OUT**

- Mỗi trọng số tối thiểu của cây khung chứa cạnh (u, v) in trên 1 dòng.

Ví dụ:

WEIGHTS.INP	WEIGHTS.OUT	Giải thích

5 7 1 2 3 1 3 1 1 4 5 2 3 2 2 5 3 3 4 2 4 5 4	9 8 11 8 8 8 9	 MST(1,2) = w(1,2)+w(3,4)+w(2,5) = 3+1+2+3=9 MST(1,3) = w(1,3)+w(2,3)+w(3,4)+w(2,5) = 1 + 2 + 2 + 3 = 8 Tương tự các cạnh khác.
2 1 1 2 42	42	

b) <u>Thuật toán</u>

Xây dựng Cây bao trùm tối thiểu (MST). Sau đó, đối với tất cả các cạnh có trong MST, kết quả sẽ là trọng số tổng của MST. Đối với các cạnh không thuộc MST, giả sử là (u, v), chúng ta sẽ loại bỏ cạnh có trọng số lớn nhất trên đường đi từ nút u đến nút v và thêm cạnh (u, v).

Để tìm cạnh có trọng số lớn nhất trên đường đi từ nút u đến nút v, ta tìm cạnh có trọng số lớn nhất từ nút u đến nút l và từ nút v đến nút l, trong đó l là $LCA(u, v)$. Tính toán trước thông tin tổ tiên và trọng số lớn nhất sử dụng *Binary Lifting* để truy vấn hiệu quả. Điều này cho phép phân tích các cạnh riêng biệt bằng cách nhanh chóng tìm LCA và điều chỉnh trọng số MST với việc xem xét trọng số lớn nhất đã được bao phủ, cuối cùng cung cấp trọng số cây bao trùm tối thiểu với mỗi cạnh.

Các bước giải quyết vấn đề:

- Xây dựng MST sử dụng Thuật toán Kruskal.
- Tiền xử lý cho các truy vấn hiệu quả:
    - Thực hiện một tìm kiếm theo chiều sâu (DFS) bắt đầu từ bất kỳ nút nào.
    - Đối với mỗi nút đã thăm, cập nhật mảng tổ tiên và mảng $maxWeight[]$ tại các cấp độ khác nhau cho tất cả các đỉnh con của nó.
    - Sử dụng *Binary Lifting* để tính toán tổ tiên và $maxWeight[]$ cho mỗi nút con.
- Xử lý truy vấn cạnh:

Đối với mỗi cạnh, kiểm tra xem cạnh đó đã được bao gồm trong MST hay chưa sử dụng mảng $included[]$.

Nếu cạnh đã có trong MST, kết quả sẽ là trọng số của MST.

Nếu không, tìm LCA của các điểm cuối của cạnh và tính trọng số MST điều chỉnh bằng cách thêm trọng số của cạnh hiện tại và trừ đi trọng số lớn nhất trên đường đi đến LCA đã được lưu trong mảng $maxWeight[\quad]$.

In ra trọng số MST đã điều chỉnh.

c) <u>Chương trình</u>

```
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 200005;
const int LOG = 20;
int n, m;
int U[MAXN], V[MAXN], W[MAXN];
int parent[MAXN], sz[MAXN];
bool inMST[MAXN];
long long totalWeight = 0;
vector<pair<int,int> > adj[MAXN];
int up[MAXN][LOG];
int depth[MAXN];
int maxEdge[MAXN][LOG];
int findSet(int u) {
   if (u == parent[u]) return u;
   return parent[u] = findSet(parent[u]);
}
void unionSet(int u, int v) {
   u = findSet(u);
   v = findSet(v);
   if (u == v) return;
   if (sz[u] < sz[v]) swap(u, v);
   parent[v] = u;
   sz[u] += sz[v];
}
void dfs(int u, int p) {
   for (int i = 0; i < adj[u].size(); i++) {
      int v = adj[u][i].first;
      int w = adj[u][i].second;
      if (v == p) continue;
      depth[v] = depth[u] + 1;
```

```
        up[v][0] = u;
        maxEdge[v][0] = w;
        for (int j = 1; j < LOG; j++) {
            up[v][j] = up[ up[v][j-1] ][j-1];
            maxEdge[v][j] = max(maxEdge[v][j-1],
                                maxEdge[ up[v][j-1] ][j-1]);
        }
        dfs(v, u);
    }
}
int getMaxEdge(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int ans = 0;
    int diff = depth[u] - depth[v];
    for (int j = 0; j < LOG; j++) {
        if (diff & (1 << j)) {
            ans = max(ans, maxEdge[u][j]);
            u = up[u][j];
        }
    }
    if (u == v) return ans;
    for (int j = LOG - 1; j >= 0; j--) {
        if (up[u][j] != 0 && up[u][j] != up[v][j]) {
            ans = max(ans, maxEdge[u][j]);
            ans = max(ans, maxEdge[v][j]);
            u = up[u][j];
            v = up[v][j];
        }
    }
    ans = max(ans, maxEdge[u][0]);
    ans = max(ans, maxEdge[v][0]);
    return ans;
}
bool cmp(int a, int b) {
    return W[a] < W[b];
}
int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
```

```
        cin >> U[i] >> V[i] >> W[i];
    }
    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        sz[i] = 1;
    }
    vector<int> id(m);
    for (int i = 0; i < m; i++)
        id[i] = i;
    sort(id.begin(), id.end(), cmp);
    for (int i = 0; i < m; i++) {
        int e = id[i];
        int u = U[e];
        int v = V[e];
        int w = W[e];
        if (findSet(u) != findSet(v)) {
            unionSet(u, v);
            inMST[e] = true;
            totalWeight += w;
            adj[u].push_back(make_pair(v, w));
            adj[v].push_back(make_pair(u, w));
        }
    }
    depth[1] = 0;
    dfs(1, 0);
    long long answer[MAXN];
    for (int i = 0; i < m; i++) {
        if (inMST[i]) {
            answer[i] = totalWeight;
        } else {
            int mx = getMaxEdge(U[i], V[i]);
            answer[i] = totalWeight - mx + W[i];
        }
    }
    for (int i = 0; i < m; i++)
        cout << answer[i] << "\n";

    return 0;
}
```

d) <u>Độ phức tạp</u>

Độ phức tạp thời gian: $O(M\ log\ M\ +\ M\ log\ N)$

Độ phức tạp không gian: $O(N\ log\ N\ +\ N\ +\ M)$

2.2 BÀI TẬP ÁP DỤNG

2.2.1 Bài 1 – Chạy trốn.

2.2.1.1 Đề bài

https://usaco.org/index.php?page=viewproblem2&cpid=213

Nông trại của John có N bãi cỏ được đánh số từ $1..N$ và được nối bằng $N-1$ con đường hai chiều. Nhà chứa nằm ở bãi cỏ thứ nhất, và từ bãi thứ nhất, ta có thể đi đến tất cả các bãi cỏ còn lại. Những con bò của John đang ở bãi cỏ của chúng vào sáng nay, nhưng không ai biết chúng đã đi đâu cho tới bây giờ. FJ biết rằng những con bò chỉ muốn chạy xa khỏi nhà chứa, nhưng cũng rất lười nên không thể chạy một đoạn đường có độ dài lớn hơn L. Với mỗi bãi cỏ, John muốn biết có bao nhiêu bãi cỏ mà những con bò bắt đầu tại bãi cỏ i có thể đến được **theo hướng đi xa khỏi nhà chứa** với tổng độ dài $\leq\ L$

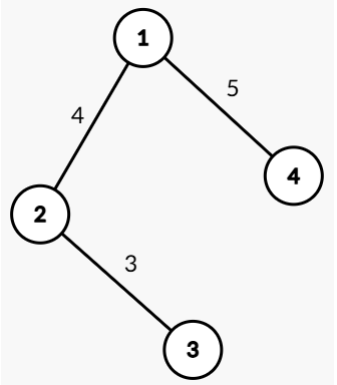
Dữ liệu vào: Cho trong file **RUNAWAY.INP**

- Dòng 1: Gồm 2 số tự nhiên N và L ($1\ \leq\ N \leq 2 \times 10^5$; $1\ \leq\ L\ \leq\ 10^{18}$).
- $N-1$ dòng tiếp theo (từ dòng 2 đến dòng N): dòng thứ i gồm hai số p_i và l_i. Trong đó p_i ($1\ \leq\ p_i\ <\ i$) thể hiện bãi cỏ i có đường nối trực tiếp với bãi cỏ p_i và có khoảng cách l_i ($1\ \leq\ l_i\ \leq\ 10^{12}$).

Kết quả ra: In ra file **RUNAWAY.OUT**

- Gồm N số nguyên, mỗi số trên một dòng, số trên dòng thứ i là số lượng các bãi cỏ có thể đi tới từ bãi cỏ thứ i bằng cách đi con đường rời xa nhà chứa với tổng độ dài không vượt quá L.

Ví dụ:

RUNAWAY.INP	RUNAWAY.OUT	Minh họa
4 5 1 4 2 3 1 5	3 2 1 1	

Giải thích: Các con bò từ bãi cỏ thứ nhất có thể trốn tại 3 bãi cỏ thứ 1,2, và 4. Các con bò từ bãi cỏ thứ hai có thể trốn tại 2 bãi cỏ là bãi thứ 2 và 3. Con bò ở bãi cỏ thứ 3 có thể trốn tại bãi số 3. Con bò ở bãi cỏ thứ 4 có thể trốn ở bãi cỏ 4.

2.2.1.2 Thuật toán

Bài toán yêu cầu: với một cây có gốc tại 1, với mỗi đỉnh v, tìm số lượng các hậu duệ của nó mà có khoảng cách đến v không vượt quá L.

Nếu ta gán cho mỗi đỉnh v một giá trị độ sâu (depth, tức là khoảng cách từ gốc đến đỉnh đó), thì với một đỉnh có độ sâu D, bài toán tương đương với việc đếm số hậu duệ có độ sâu không vượt quá $D + L$.

Cách đơn giản và ngắn gọn nhất để giải bài này là "lật ngược" tư duy bài toán:

Với mỗi đỉnh v, tìm tổ tiên cao nhất (ancestor) sao cho khoảng cách từ v đến tổ tiên đó không vượt quá L.

Cách làm này có thể thực hiện như sau:

DFS kết hợp tìm kiếm nhị phân trên stack: Khi duyệt DFS, ta giữ lại một stack gồm các tổ tiên của đỉnh hiện tại (theo thứ tự trên cây). Với mỗi đỉnh, ta có thể tìm kiếm nhị phân trên stack để tìm tổ tiên xa nhất thỏa khoảng cách $\leq L$.

Hoặc sử dụng kĩ thuật $binary\ lifting$: Tạo bảng $par[i][j]$ để nhảy lên tổ tiên của i cách 2^j bước, từ đó có thể nhảy lên nhanh chóng từ một đỉnh để tìm tổ tiên xa nhất trong giới hạn khoảng cách.

Chuyển ngược kết quả về dạng hậu duệ. Sau khi đã xác định với mỗi đỉnh v, đỉnh tổ tiên xa nhất u mà khoảng cách từ v đến u không vượt quá L. Ta có thể đánh dấu u là vị trí mà v không được tính là hậu duệ nữa (vì vượt quá L). Sau đó, với mỗi đỉnh, ta tính tổng số hậu duệ trong phạm vi khoảng cách có thể đến được.

Cộng tổng kết quả từ các bãi cỏ con lên các bãi cỏ cha. Trừ đi số lần bị đánh dấu tại đỉnh đó.

2.2.1.3 Đánh giá

Bài toán yêu cầu hiểu sâu về cấu trúc cây, đặc biệt là kĩ thuật *binary lifting*, và cách tối ưu các phép tính tổ tiên trong cây. Đây là dạng bài ứng dụng của bài toán mô tả trong phần 2.1.1 (ANCESTOR)

- Độ phức tạp thời gian: $O(nlogn)$
- Độ phức tạp không gian: $O(nlogn)$

2.2.2 Bài 2 – Đoạn con không giao nhau có tổng bằng 0

2.2.2.1 Đề bài

https://oj.uz/problem/view/RMI20_sumzero

Cho một mảng gồm N số nguyên: $c_1, c_2, \ldots, c_N$ và Q cặp (L_i, R_i), mỗi cặp đại diện cho một đoạn con của mảng: c_{L_i}, $c_{L_{i+1}}$, ..., c_{R_i} trong đó $1 \leq i \leq N$.

Với mỗi cặp (L_i, R_i), đếm số lượng lớn nhất các đoạn con không giao nhau trong đoạn được truy vấn c_{L_i}, $c_{L_{i+1}}$, ..., c_{R_i} mà có tổng bằng 0. Hai đoạn con được xem là không giao nhau nếu chúng không có phần tử chung; tuy nhiên, chúng vẫn có thể kề nhau tại điểm đầu hoặc cuối. Lưu ý rằng có thể tồn tại các phần tử trong đoạn được truy vấn không thuộc về bất kỳ đoạn con nào được chọn.

Dữ liệu vào: Cho trong file **SUMZERO.INP**

- Dòng đầu tiên chứa một số nguyên duy nhất N là độ dài của mảng.
- Dòng thứ hai chứa N số nguyên cách nhau bởi dấu cách: $c_1, c_2, \ldots, c_N$.
- Dòng thứ ba chứa số nguyên Q – số lượng truy vấn.
- Q dòng tiếp theo, mỗi dòng chứa hai số nguyên (L_i, R_i), đại diện cho truy vấn thứ i.

Kết quả ra: In trong file **SUMZERO.OUT**

- In ra Q dòng, mỗi dòng chứa câu trả lời cho truy vấn tương ứng – số lượng lớn nhất các đoạn con không giao nhau có tổng bằng 0 trong đoạn con được chỉ định.

Ví dụ:

SUMZERO.INP	SUMZERO.OUT
10 1 2 -3 0 1 -4 3 2 -1 1 3 1 10 1 5 2 9	4 2 2

Giới hạn:

Subtask 1: 20 % số test có: $1 \leq N, Q \leq 5000$

Subtask 2: 40% số test có: $1 \leq N, Q \leq 10^5$

Subtask 3: 40 % số test có:

$$1 \leq N, Q \leq 4 \times 10^5$$

$$-10^9 \leq c_i \leq 10^9 \text{ với mọi } 1 \leq i \leq N$$

$$1 \leq L_i \leq R_i \leq N \text{ với mọi } 1 \leq i \leq Q$$

2.2.2.2 Thuật toán

Để trả lời truy vấn trong đoạn từ l đến r ta phải tìm được nhiều nhất các đoạn có tổng bằng 0

Gọi $pre[i]$ là tổng tiền tố đến i

Ta có : $pre[r] - pre[l-1] == 0 \rightarrow pre[r] == pre[l-1]$

Vậy nên có thể hiểu bài toán theo cách khác là đếm số cặp $pre[r] == pre[l-1]$ trong đoạn L, R;

Áp dụng kĩ thuật $binary\ lifting$: xây dựng mảng $nxt[i][j]$ với ý nghĩa là vị trí tiếp theo của phần tử i sau khi qua 2^j lần để tìm được cặp. Mỗi bước nhảy tương ứng với việc đi từ một chỉ số đến chỉ số xa hơn trong mảng theo một khoảng cách gấp đôi (bước nhảy nhị phân).

Sau khi xây dựng bảng nxt, chương trình sẽ xử lý các truy vấn. Mỗi truy vấn yêu cầu tính toán số lượng bước nhảy có thể thực hiện từ chỉ số l đến r sao cho tổng các phần tử trong đoạn con từ l đến r thỏa mãn điều kiện.

2.2.2.3 Đánh giá

Bài toán tìm kiếm chỉ số trong một dãy số sao cho các phép tìm kiếm tối ưu. Độ phức tạp của từng phần chương trình:

- Tính tổng tiền tố: $O(n)$
- Sắp xếp mảng b theo tổng tiền tố và cập nhật chỉ số mảng a: $O(nlogn)$
- Kĩ thuật $binary\ lifting$ (tạo bảng $nxt[i][j]$): $O(nlogn)$
- Truy vấn: Mỗi truy vấn yêu cầu duyệt qua bảng nhảy để tìm giá trị tối ưu mất $O(logn)$

Tổng độ phức tạp của bài toán: $O(nlogn + qlogn)$

2.2.3 Bài 3 – Đoạn tốt liên tiếp

2.2.3.1 Đề bài

https://www.codechef.com/problems/LGSEG?tab=statement

Cho một dãy $A_1, A_2, \dots, A_N$. Gọi một dãy con liên tiếp của A là một đoạn.

Một đoạn là tốt nếu có thể chia nó thành K đoạn sao cho tổng các phần tử của mỗi đoạn con tối đa là S.

Bạn cần tìm số phần tử tối đa trong một đoạn tốt.

Dữ liệu vào: Cho trong file **LGSEG.INP**

- Dòng đầu tiên của dữ liệu vào chứa một số nguyên T thể hiện số lượng test. T test được miêu tả như sau:
    - Dòng đầu tiên của mỗi test chứa ba số nguyên N, K và S

- Dòng thứ hai chứa N số nguyên $A_1, A_2, \dots, A_N$

Kết quả ra: In ra file **LGSEG.OUT**

- Với mỗi test, in ra một dòng chứa một số nguyên – độ dài lớn nhất của một đoạn tốt.

Giới hạn:

- $1 \le T \le 15 \times 10^4$
- $1 \le K \le N \le 10^5$
- $1 \le S \le 10^9$
- $1 \le A_i \le S$ với mọi i
- Tổng của N trong tất cả các test không vượt quá 10^6

Ví dụ.

LGSEG.INP	LGSEG.OUT	Giải thích
2 5 2 5 1 3 2 1 5 5 3 5 5 1 5 1 1	4 4	Ví dụ 1: Một trong những đoạn có độ dài 4 là [1, 3, 2, 1]. Nó có thể chia thành hai đoạn [1, 3] và [2, 1] có tổng đối đa S = 5.

2.2.3.2 Thuật toán

Ý tưởng duyệt: Ta sẽ cố định độ dài, sau đó duyệt từng đoạn có độ dài đó và kiểm tra xem có chia được thành k đoạn hợp lệ hay không. Với cách này chúng ta sẽ có độ phức tạp là $O(n^2 log(n))$.

Vậy ta sẽ cải tiến phần check bằng kĩ thuật *binary lifting*

Cách làm:

- Ta sẽ chuẩn bị mảng $Next[i]$: vị trí đầu tiên tính từ i mà $sum(i\,..Next[i] - 1) \le S$, nhưng $sum(i\,..Next[i] - 1) > S$. Ta có thể xây dựng mảng này bằng kĩ thuật 2 con trỏ hoặc chặt nhị phân.
- Tiếp theo ta cần xây dựng mảng $up[t][i]$: vị trí kết thúc tính từ i sau khi ta nhảy qua 2^t đoạn con có tổng không vượt quá S. Để xây dựng được mảng up, ta sử dụng kĩ thuật tương tự như LCA. Với 2 công thức chính như sau:

$$up[0][i] = Next[i];$$

$$up[t][i] = up[t - 1][i <= n\,?\, up[t - 1][i] : n + 1];$$

Vậy từ đó, chúng ta có thể viết hàm *check* dựa theo mảng *up* vừa được tính. Ở đây chúng ta sẽ sử dụng kĩ thuật *binary lifting*. Khi xét độ dài len, chúng ta sẽ duyệt qua từng vị trí trong mảng, tại mỗi vị trí, ta duyệt từ $log(n)$ về 0. Tại một vị trí đang xét, ta sẽ nhảy đến $up[t][pos]$, hay chính là nhảy qua 2^t đoạn thỏa mãn có tổng không quá S. Sau đó kiểm tra xem vị trí đó có thuộc đoạn $[i, i + len - 1]$ hay không và tính xem ta đã sử

dụng bao nhiêu đoạn khi đến đó. Cuối cùng nếu tại một vị trí i nào đó mà số đoạn ta dùng không quá k thì ta sẽ trả về $true$. Ngược lại là $false$.

2.2.3.3 Đánh giá

Bài toán này sử dụng kĩ thuật *Binary Lifting* để tối ưu hóa quá trình tìm độ dài tối đa của đoạn con sao cho tổng các phần tử của đoạn này không vượt quá S và số lần sử dụng đoạn con không quá k.

Tiền xử lý: Mảng up để lưu trữ chỉ số tiếp theo thỏa mãn điều kiện. Mảng $pre[i] = a[1] + a[2] + \ldots + a[i]$ giúp tính tổng của bất kì đoạn con nào trong mảng 1 cách nhanh chóng. Phần này mất độ phức tạp: $O(N\ log\ N)$

Mỗi lần kiểm tra độ dài đoạn con, sử dụng *binary lifting* để duyệt mảng và tìm chỉ số tiếp theo sao cho tổng không vượt quá S có độ phức tạp $O(log\ N)$

Do đó, tổng độ phức tạp của mỗi truy vấn là $O(N\ log\ N)$, và với T test case, độ phức tạp tổng thể là $O(T\ *\ N\ log\ N)$.

2.2.4 Bài 4 – Trò chơi 262144

https://usaco.org/index.php?page=viewproblem2&cpid=648

2.2.4.1 Đề bài

Cô bò Bessie thích tải trò chơi về điện thoại di động của mình, mặc dù với đôi móng to lớn, màn hình cảm ứng nhỏ khiến cô hơi vất vả để điều khiển. Trò chơi hiện tại bắt đầu với một dãy gồm N số nguyên dương ($2 \leq N \leq 262144$) mỗi số nằm trong khoảng từ 1 đến 40. Trong một nước đi, Bessie có thể lấy hai số **kề nhau** nếu chúng **bằng nhau**, và thay chúng bằng một số mới có giá trị lớn hơn một đơn vị (ví dụ: hai số 7 liền nhau sẽ thành một số 8). Mục tiêu là làm sao để giá trị của số lớn nhất trong dãy cuối cùng là **cao nhất** có thể. Hãy giúp Bessie đạt được điểm cao nhất!

Dữ liệu vào: Cho trong file **GAME.INP**

- Dòng đầu tiên gồm số nguyên dương N
- N dòng tiếp theo gồm N số để bắt đầu trò chơi.

Dữ liệu đầu ra: In ra file **GAME.OUT**

- Gồm kết quả lớn nhất mà Bessie có thể tìm được.

Ví dụ:

GAME.INP	GAME.OUT
4 1 1 1 2	3

2.2.4.2 Thuật toán

Ta có thể dùng quy hoạch động để giải quyết bài toán:

$dp[p][i] = j$: lưu vị trí j nếu có thể tạo ra số p bằng cách gộp các số trong đoạn từ i đến j. Nếu không có cách gộp thì bằng -1. Để biết xem có thể tạo được giá trị $p + 1$ từ vị trí i hay không, ta có công thức cơ bản như sau:

$$dp[p + 1][i] = max\{dp[p + 1][i], dp[p][k + 1]\}$$

(với mọi k trong $[i..j]$ sao cho $dp[p][i] = k$ và $dp[p][k + 1] = j$)).

Cách trên có độ phức tạp $O(maxa * n^2)$. Quá chậm với n lớn như vây. Do đó, để tối ưu ta có thể áp dụng *binary lifting* để tối ưu hơn bằng công thức:

$$dp[p + 1][i] = dp[p][dp[p][i]] \ (dp[p][dp[p][i]] \neq -1).$$

Công thức này đúng và tối ưu bởi khi đã biết $dp[p][i]] = mid$ (tức là đoạn từ $[i \ldots mid]$ có thể tạo ra p thì để tạo ra số $p + 1$, ta sẽ gộp 2 đoạn liên tiếp có giá trị đều bằng p mà bắt đầu từ $(i \rightarrow mid)$ và bắt đầu từ mid.

Dễ thấy rằng nếu 2 số $a[i] = a[i + 1]$ thì khi tạo ra số $(a[i] + 1)$, dãy sẽ bị mất đi 1 phần tử. Và dần dần khi tạo được số lớn nhất thì mỗi lần dãy sẽ bị giảm đi 1 lượng khá lớn phần tử theo cấp số nhân. Do đó, số lớn nhất mà dãy có thể tạo được đó là $(maxa + logN) \approx (40 + 18) = 58$.

2.2.4.3 Đánh giá

- Độ phức tạp $O(N * 58)$

2.2.5 Bài 5 – Truy vấn trên cây

2.2.5.1 Đề bài

https://www.acmicpc.net/problem/32973

Cho một cây không trọng số có N đỉnh, trong đó gốc của cây là đỉnh 1. Mỗi đỉnh i có một giá trị gán sẵn $v[i]$. Cây được mô tả bằng mảng $p[i]$, với mỗi i từ 1 đến $N - 1$, $p[i]$ là cha của đỉnh $i + 1$.

Định nghĩa hàm $f(y)$ cho một đỉnh y như sau:

$$f(y) = \sum_{x \in S_y} d(x, y) \cdot v[x]$$

trong đó $d(x, y)$ là khoảng cách giữa hai đỉnh x và y, và S_y là tập tất cả các đỉnh có y là một tổ tiên.

Cho Q truy vấn, mỗi truy vấn gồm hai đỉnh x và y. Với mỗi truy vấn, bạn phải thực hiện các bước biến đổi tạm thời trên cây như sau, sau đó tính giá trị $f(y)$ trên cây đã thay đổi:

1) Gắn tất cả các con của nút x (các đỉnh có cha là x) vào cha của x.

2) Loại bỏ nút x ra khỏi cây.

3) Chèn nút x trở lại cây, giữa nút y và một trong các con của y từ trong cây con của y mà x đã bị loại bỏ.

Nếu y là cha của x, cây không thay đổi. Luôn đảm bảo rằng x nằm trong cây con của y. Sau mỗi truy vấn, sau khi tính $f(y)$, cây sẽ được khôi phục về trạng thái ban đầu (các thay đổi chỉ tạm thời).

Dữ liệu vào: Cho trong file **STABLO.INP**

- Dòng đầu tiên chứa hai số nguyên N và Q $(1 \leq N, Q \leq 5 \times 10^5)$: số đỉnh và số truy vấn.
- Dòng thứ hai chứa N số nguyên $v[i]$ $(1 \leq v[i] \leq 10^6)$: giá trị của mỗi đỉnh từ 1 đến N.
- Dòng thứ ba chứa $N-1$ số nguyên $p[i]$ $(1 \leq p[i] \leq i)$: với mỗi i từ 1 đến $N-1, p[i]$ là cha của đỉnh $i+1$.
- Mỗi dòng trong Q dòng tiếp theo chứa hai số x và y $(1 \leq x, y \leq N)$: chỉ ra truy vấn cần thực hiện như mô tả.

Kết quả ra: In ra file **STABLO.OUT**

- In ra Q dòng, mỗi dòng là giá trị của $f(y)$ trên cây đã biến đổi theo truy vấn tương ứng.

Ví dụ

STABLO.INP	STABLO.OUT	Giải thích
3 1 1 2 3 1 2 3 1	7	Sau khi thực hiện truy vấn, nút 3 cách nút 1 một cạnh, nút 2 cách nút 1 hai cạnh. Do đó: $f(1) = 0 + 2 \cdot 2 + 1 \cdot 3 = 7$.
3 2 4 5 6 1 1 2 1 3 1	11 11	
5 3 2 5 2 2 2 1 2 3 2 4 3 3 2 5 1	2 8 26	

Giới hạn

Subtask	Điểm	Ràng buộc bổ sung
1	21	$1 \leq N, Q \leq 1000$
2	37	Cây là một đường thẳng, p[i]=i với mọi i từ 1 đến N−1.

3	22	Mỗi nút có nhiều nhất 20 con.
4	40	Không có ràng buộc thêm.

2.2.5.2 Thuật toán

Hàm $f(y)$ được định nghĩa như sau: Nhân giá trị của mỗi nút với hệ số tương ứng (khoảng cách đến y) rồi cộng vào $f(y)$.

Subtask 1:

Với mỗi truy vấn, ta có thể xây dựng lại cây và tính trực tiếp $f(y)$. Độ phức tạp sẽ là $O(n \times q)$.

Mỗi thao tác trên cây có thể được xem là một sự thay đổi hệ số thay vì kết nối và kết nối lại các nút. Điều này đặc biệt hữu ích vì thao tác với hệ số đơn giản hơn so với việc kết nối lại các nút và cây con.

Định lý (Claim 1): Để trả lời bất kỳ truy vấn nào, chúng ta phải có thể:

- Tính được $f(y)$ cho mọi nút y.
- Tăng hoặc giảm hệ số trong cây con của một nút.
- Tìm con trực tiếp của y mà là tổ tiên của x.

Chứng minh: Giả sử chúng ta muốn mô phỏng ba bước biến đổi của truy vấn:

1) Tính $f(y)$ trên cây ban đầu.
2) Giảm hệ số trong cây con của x đi 1 (tương ứng bước "tháo" nút x).
3) Tìm con của y mà là tổ tiên của x, rồi tăng hệ số trong cây con đó lên 1 (tương ứng bước "chèn" lại vào con của y).
4) Cuối cùng, giảm hệ số của riêng nút x đi 1 (để loại trừ chính nó khỏi cây con mới của y).

Mỗi bước trên đều sử dụng đúng một trong ba thao tác đã nêu, nên đủ để thực hiện toàn bộ truy vấn.

Triển khai các thao tác:

- Điều kiện tăng hoặc giảm các hệ số trong cây con của một nút tương đương với việc cộng hoặc trừ tổng giá trị của các nút trong cây con. Gọi hàm $g(y)$ đại diện cho tổng giá trị trong cây con của y. Chúng ta có:

$$g(y) = vl[y] + \sum g(x), \qquad par[x] = y$$

- Tính hàm $f(y)$ có thể thực hiện theo nhiều cách, nhưng một trong những cách đơn giản nhất là:

$$f(y) = g(y) - vl[y] + \sum f(x), par[x] = y$$

(tổng qua tất cả con x của y).

- Tìm con của y là tổ tiên của x bằng phương pháp nâng nhị phân (binary lifting) để tìm nhanh vị trí z là con của y sao cho z cũng là tổ tiên của x.

Sử dụng bảng $up[x][k]$ như trình bày tại phần 2.1.1 để tìm tổ tiên mức 2^kcủa nút x sao cho $2^k \leq d$ (d là độ sâu của tổ tiên cần tìm)

- Nếu x có độ sâu là d_x và y có độ sâu là d_y , bạn muốn tìm tổ tiên của x sao cho độ sâu của tổ tiên này là d_y. Di chuyển x lên theo các mức tổ tiên từ bảng $binary\ lifting$ cho đến khi độ sâu của tổ tiên đạt được d_y.

- Duyệt các mức tổ tiên từ $j = logn$ xuống $j = 0$. Ở mỗi mức j, nếu độ sâu d_x của x là lớn hơn hoặc bằng d_y+2^j, di chuyển x lên tổ tiên tại mức j. Quá trình này sẽ giúp ta "nhảy" lên các tổ tiên của x cho đến khi đạt được độ sâu yêu cầu.

- Ví dụ, nếu d_x−d_y= 7, ta sẽ di chuyển từ mức 2^3 xuống mức 2^0 bằng cách lần lượt kiểm tra bảng tổ tiên và di chuyển x lên các tổ tiên theo mỗi mức.

2.2.5.3 Đánh giá

Độ phức tạp để thực hiện một lần DFS để tính trước mọi giá trị $g(y)$ và $f(y)$cho cây gốc là $O(n)$.

Với mỗi truy vấn, ta làm một số thao tác cập nhật trên các cây con và một phép tìm nhị phân là $O(logn)$.

Tổng độ phức tạp là: $O((n + q)\ logn)$.

2.3 BÀI TẬP TỰ LUYỆN

2.3.1 Bài 1 - Cắt đoạn

2.3.1.1 Đề bài

https://codeforces.com/problemset/problem/1516/D

Cho một mảng a có n số nguyên. Thực hiện các thao tác sau:

(1) Chọn một khoảng (l, r) và cắt đoạn con $a[l], a[l + 1], \dots, a[r]$ ra, loại bỏ phần còn lại của mảng.

(2) Sau đó cắt đoạn này thành nhiều đoạn con sao cho các phần tử của mỗi đoạn con phải có tích của chúng bằng với bội chung nhỏ nhất (LCM) của chúng.

Cụ thể là phân chia các phần tử của $a[l], a[l + 1], \dots, a[r]$ thành các mảng con liên tiếp sao cho tích của mỗi mảng con bằng với LCM của nó.

Cho q truy vấn (l, r), hãy cho biết số lượng mảng con tối thiểu thỏa mãn.

Dữ liệu vào: Cho trong file **CUT.INP**

- Dòng đầu tiên chứa 2 số nguyên n và q $(1 \leq n, q \leq 10^5)$ – độ dài của mảng a và số lượng truy vấn.
- Dòng tiếp theo chứa n số nguyên $a_1, a_2, \dots, a_n$ $(1 \leq a_i \leq 10^5)$ – các phần tử của mảng a.
- q dòng tiếp theo, mỗi dòng chứa 2 số nguyên l và r $(1 \leq l \leq r \leq n)$

Kết quả ra: Cho trong file **CUT.OUT**

Đối với mỗi truy vấn, in câu trả lời của nó trên một dòng mới.

Ví dụ:

CUT.INP	CUT.OUT
6 3 2 3 10 7 5 14 1 6 2 4 3 5	3 1 2

Giải thích:

Truy vấn đầu tiên hỏi về toàn bộ mảng. Bạn có thể phân chia nó thành $[2]$, $[3,10,7]$, và $[5,14]$. Đoạn con đầu tiên có tích và LCM bằng 2. Đoạn thứ hai có tích và LCM bằng 210. Và khoảng thứ ba có tích và LCM bằng 70. Một phân chia khả thi khác là $[2,3]$, $[10,7]$, và $[5,14]$.

Truy vấn thứ hai hỏi về khoảng $(2,4)$. Tích của nó bằng với LCM của nó, vì vậy bạn không cần phải phân chia thêm.

Truy vấn cuối cùng hỏi về khoảng $(3,5)$. Phân chia thành $[10,7]$ và $[5]$.

2.3.1.2 Thuật toán

Hãy cùng hiểu ý nghĩa của **"tích = LCM"**. Xét một số nguyên tố p. Khi thực hiện phép nhân, số mũ của p trong tất cả các số sẽ được cộng lại; còn với LCM, ta lấy số mũ lớn nhất. Do đó, hai giá trị này chỉ bằng nhau nếu mỗi số nguyên tố chia hết cho nhiều nhất một số trong đoạn. Một cách nghĩ khác là mọi cặp số đều nguyên tố cùng nhau.

Ta có thuật toán tham lam như sau: giả sử ta bắt đầu từ chỉ số l; ta sẽ tiếp tục mở rộng đoạn con đầu tiên miễn là điều kiện (mọi cặp số nguyên tố cùng nhau) còn thỏa mãn. Rõ ràng ta không được lợi gì khi dừng lại nếu còn mở rộng được, vì mỗi phần tử mới chỉ mang đến những ràng buộc mới. Khi không thể mở rộng thêm, ta bắt đầu đoạn con mới, cho đến khi đến chỉ số r.

Với mỗi chỉ số l, ta định nghĩa go_l là chỉ số đầu tiên khiến điều kiện bị phá vỡ nếu ta thêm phần tử đó vào đoạn con. Thuật toán tương đương với việc bắt đầu tại chỉ số $cur = l$, rồi thay cur bằng go_{cur} cho đến khi vượt qua chỉ số r. Số bước thực hiện là đáp án.

Có hai bài toán con:

1. Tính giá trị go_l:

Ta sẽ duyệt mảng a từ cuối về đầu. Tại chỉ số l, ta duyệt các ước nguyên tố của $a[l]$. Với mỗi ước nguyên tố, ta tìm phần tử tiếp theo chia hết cho nó. Ta có thể lưu chỉ số đó trong một mảng và cập nhật dần. Nếu ta lấy giá trị nhỏ nhất trong các chỉ số này, ta sẽ tìm được phần tử tiếp theo không nguyên tố cùng nhau với l. Gán go_l = chỉ số đó. Tuy nhiên,

nếu hai phần tử khác (không bao gồm l) không nguyên tố cùng nhau thì sao? Cách khắc phục thông minh là ta lấy giá trị nhỏ hơn giữa go_l và go_{l+1}, vì go_{l+1} đã bao gồm tất cả các phần tử sau l.

2. Đếm số bước nhanh:

Đây là một bài toán khá tiêu chuẩn, có thể giải bằng kĩ thuật "nhảy nhị phân" (binary lifting). Ý tưởng là thực hiện nhiều bước nhảy cùng lúc thay vì từng bước một. Ta tính $dp[i][l]$: chỉ số ta đến được nếu thực hiện 2^i bước từ chỉ số l.

Rõ ràng, $dp[i][l] = dp[i-1][dp[i-1][l]]$ vì $2^{i-1} + 2^{i-1} = 2^i$.

Giờ để tính số bước từ chỉ số l đến chỉ số r, ta duyệt từ $log(n)$ về 0. Với mỗi i, nếu $dp[i][l] \leq r$, ta có thể nhảy 2^i bước, nên ta cập nhật $l = dp[i][l]$ và cộng 2^i vào đáp án. Cuối cùng, ta thực hiện thêm một bước nhảy nữa.

2.3.1.3 Đánh giá

Bài toán có độ khó khá cao và yêu cầu kết hợp nhiều kĩ thuật tối ưu như sàng nguyên tố, phân tích thừa số nguyên tố và kĩ thuật *binary lifting*.

Độ phức tạp chủ yếu là $O(nlogn)$ cho việc chuẩn bị và $O(log\ n)$ cho mỗi truy vấn. Do vậy tổng độ phức tạp của bài toán là: $O(nlogn + qlogn)$

2.3.2 Bài 2 – Dịch vòng hoán vị

2.3.2.1 Đề bài

https://codeforces.com/contest/1142/problem/B

Cho 1 hoán vị p độ dài n và 1 mảng a có m phần tử gồm các số nguyên đánh chỉ số từ 1 tới n.

Cho q truy vấn, mỗi truy vấn cần trả lời câu hỏi: "có phải đoạn con từ l đến r của a (bao gồm cả a_l và a_r có một chuỗi con là dịch vòng của p không"

Một *hoán vị* có độ dài n là một chuỗi gồm n số nguyên sao cho mỗi số nguyên từ 1 đến n xuất hiện chính xác một lần trong đó.

Một *dịch vòng* của một hoán vị $(p_1, p_2, \dots, p_n)$ là một hoán vị $(p_i, p_{i+1}, \dots, p_n, p_1, p_2, \dots, p_{i-1})$ với mọi i từ 1 đến n. Ví dụ, một hoán vị (2,1,3) có ba dịch vòng khác nhau: (2,1,3), (1,3,2), (3,2,1).

Một *chuỗi con* của một đoạn con của mảng a từ vị trí thứ l đến vị trí thứ r là một chuỗi $(a_{i_1}, a_{i_2}, \dots, a_{i_k})$ với $l \leq i_1 < i_2 < \dots < i_k \leq r$

Dữ liệu vào: Cho trong file **PERMUT.INP**

- Dòng đầu tiên chứa ba số nguyên n, m, q $(1 \leq n, m, q \leq 2 \times 10^5)$ - độ dài của hoán vị p, độ dài của mảng a và số lượng truy vấn.
- Dòng tiếp theo chứa hoán vị p bao gồm các số nguyên từ 1 đến n, mỗi số xuất hiện chính xác một lần.

- Dòng tiếp theo chứa m số nguyên từ 1 đến n là các phần tử của mảng a
- q dòng tiếp theo mô tả các truy vấn. Dòng thứ i trong số này chứa hai số nguyên l_i và r_i $(1 \le l_i \le r_i \le m)$

Kết quả ra: In ra file **PERMUT.OUT**

In ra một chuỗi duy nhất có độ dài q, bao gồm 0 và 1, là 1 nếu đoạn con của mảng a từ vị trí l_i và r_i chứa một chuỗi con là một dịch vòng của p, là 0 nếu không chứa chuối dịch vòng của p.

Ví dụ

PERMUT.INP	PERMUT.OUT
3 6 3 2 1 3 1 2 3 1 2 3 1 5 2 6 3 5	110
2 4 3 2 1 1 1 2 2 1 2 2 3 3 4	010

Giải thích

Trong ví dụ đầu tiên, đoạn từ vị trí thứ 1 đến vị trí thứ 5 là 1,2,3,1,2. Có một chuỗi con 1,3,2 là một dịch vòng của hoán vị. Đoạn con từ vị trí thứ 2 đến vị trí thứ 6 cũng chứa một chuỗi con 2,1,3 bằng với hoán vị. Đoạn con từ vị trí thứ 3 đến vị trí thứ 5 là 3,1,2 chỉ có một chuỗi con có độ dài 3 (3,1,2), nhưng nó không phải là một dịch vòng của hoán vị.

Trong ví dụ thứ hai, các dịch vòng có thể là 1,2 và 2,1. Đoạn con từ vị trí thứ 1 đến vị trí thứ 2 là 1,1, các chuỗi con của nó không phải là dịch vòng của hoán vị. Đoạn con từ vị trí thứ 2 đến vị trí thứ 3 là 1, 2 nó trùng với hoán vị. Đoạn con từ vị trí thứ 3 đến vị trí thứ 4 là 2,2 các chuỗi con của nó không phải là dịch vòng của hoán vị.

2.3.2.2 Thuật toán

Với mỗi a_i, nếu số a_i có vị trí j trong p, hãy tìm chỉ số lớn nhất l sao cho $l < i$ và $a_l = p_{j-1}$ ($p_0 = p_n$). Gọi vị trí này là b_i. Việc này có thể thực hiện trong thời gian $O(n)$, chỉ cần với mỗi p_j ta lưu vị trí xuất hiện cuối cùng của nó trong a khi duyệt qua a.

Với mỗi i ta có thể tìm điểm bắt đầu của dãy con bên phải nhất trong a mà là một chuỗi dịch vòng của p và kết thúc chính xác tại a_i. Điều này dễ dàng thực hiện vì nếu tồn tại một dãy con có dạng: $a_{i_1}, a_{i_2}, \dots, a_{i_{n-1}}, a_i$ là dãy con bên phải nhất thì $i_{n-1} = b_i, i_{n-2} = b_{i-1}$,... Để tìm dãy con như vậy và vị trí bắt đầu của nó, ta cần tính $b[b[b[\dots[b[i]]\dots]]]$ $n-1$ lần.

Để làm việc này, ta có thể sử dụng kĩ thuật $binary\ lifting$. Khi đó tiền xử lý trong trong thời gian $O(mlogn)$ và tìm vị trí bắt đầu của dãy con trong thời gian $O(logn)$.

Với mỗi tiền tố (prefix) của a, ta sẽ tính điểm bắt đầu của dãy con bên phải nhất của nó mà là một chuỗi dịch vòng của p. Việc này có thể thực hiện trong thời gian tuyến tính, đầu tiên ta xét kết quả của prefix mà không bao gồm phần tử cuối cùng, sau đó cập nhật lại với dãy con bên phải nhất kết thúc tại phần tử cuối của prefix.

Cuối cùng, ta có thể trả lời mỗi truy vấn trong thời gian $O(1)$, vì ta chỉ cần tìm điểm bắt đầu của dãy con bên phải nhất kết thúc tại prefix có độ dài r và so sánh nó với l.

Đánh giá

Bài toán có độ khó trung bình, yêu cầu xử lý dãy con liên tiếp - một bài toán cổ điển trong lý thuyết đồ thị và chuỗi**.** Áp dụng *Binary Lifting* giúp tối ưu việc tìm các chỉ số tiếp theo trong $log(n)$ ở mỗi bước giúp giải quyết được các mảng có kích thước lớn.

Tổng độ phức tạp: $O(n + m\ logn + q)$

2.3.3 Bài 3 – Cuộc đối chiếu

2.3.3.1 Đề bài

Trong một cuộc điều tra, đã xác định được n nghi phạm, công an đề nghị các nhân chứng tìm ra thủ phạm. Chiều cao của mỗi nghi phạm i là một số thực trong khoảng từ l_i đến r_i. Có thể có ít nhất một nghi phạm là thủ phạm, nhưng cũng có thể không có nghi phạm nào.

Một cuộc đối chiếu gồm việc chọn hai số nguyên dương a và b $(1 \leq a \leq b \leq n)$, sau đó đưa các nghi phạm từ $a, a + 1, \ldots, b$ vào một phòng riêng để các nhân chứng có thể cố gắng nhận diện thủ phạm. Do những người làm chứng có thể bị nhầm lẫn nếu hai nghi phạm có chiều cao giống nhau, nên trong một cuộc đối chiếu không có hai nghi phạm nào có chiều cao giống nhau.

Điều tra viên trưởng hiện đang quan tâm đến việc trả lời các câu hỏi sau: "Nếu nhãn của thủ phạm chỉ có thể nằm trong khoảng từ p đến q (p ≤ q), thì số lượng cuộc đối chiếu tối thiểu là bao nhiêu để nhân chứng có thể tìm ra thủ phạm trong trường hợp xấu nhất?" Hãy giúp điều tra viên trưởng trả lời q câu hỏi như vậy.

Dữ liệu vào: Cho trong file **LINEUP.INP**

Dòng đầu tiên chứa một số nguyên dương n, số lượng nghi phạm.

n dòng tiếp theo chứa hai số nguyên dương l_i và r_i $(1 \leq l_i < r_i \leq 10^9)$, đại diện cho khoảng chiều cao có thể có của nghi phạm có thứ i.

Dòng tiếp theo chứa một số nguyên dương q là số lượng câu hỏi.

q dòng tiếp theo chứa hai số nguyên dương p_i và q_i $(1 \leq p_i \leq q_i \leq n)$ là một câu hỏi.

Dữ liệu đầu vào: Cho trong file **LINEUP.OUT**

In ra q dòng, mỗi dòng là câu trả lời cho câu hỏi tương ứng: số lượng cuộc đối chiếu tối thiểu

Ví dụ:

LINEUP.INP	LINEUP.OUT
2 1 1 1 1 2 2 1 2	1 1 2
3 1 1 2 2 3 3 2 1 3 2 3	1 1
5 1 1 2 2 3 3 4 4 5 5 3 1 3 1 4 3 5	1 3 1

Giới hạn

Subtask 1: 20% số test có $q = 1, pi = 1, q1 = n$

Subtask 2: 20% số test có $1 \leq n \leq 5000, 1 \leq q \leq 5000$

Subtask 3: 20% số test có $1 \leq n \leq 5000, 1 \leq q \leq 200000$

Subtask 4: 20% số test có $1 \leq n \leq 200000, 1 \leq q \leq 5000$

Subtask 5: 20% số test có $1 \leq n, q \leq 200{,}000$

Giải thích ví dụ thứ ba:

Đối với câu hỏi thứ nhất và thứ ba, chỉ cần có ba cuộc đối chiếu: một cuộc đối chiếu gồm nghi phạm 1, một cuộc đối chiếu gồm nghi phạm 2 và một cuộc đối chiếu gồm nghi phạm 3, 4, 5.

Đối với câu hỏi thứ hai, chỉ cần có một cuộc đối chiếu gồm nghi phạm 3, 4, 5.

2.3.3.2 Thuật toán

Với dãy các khoảng đo [li, ri] của các nghi phạm, ta cần "phân chia" các nghi phạm thành các cuộc đối chiếu (lineup) sao cho trong mỗi cuộc đối chiếu không có hai nghi phạm nào có khả năng có cùng chiều cao (tức là các khoảng đo của chúng không "giao nhau").

Một cách hiệu quả để xác định số cuộc đối chiếu tối thiểu cần dùng là tính cho mỗi vị trí i một giá trị $pre[i]$. Giá trị này được tính dựa trên thao tác loại bỏ dần các nghi phạm cho đến khi các khoảng đo không giao nhau (sử dụng Segment Tree).

Cụ thể, sau khi thực hiện các cập nhật (update) trên cây ST, ta duyệt các nghi phạm từ 1 đến n, và đối với mỗi i ta cập nhật $pre[i]$ sao cho đảm bảo rằng tập các nghi phạm từ $pre[i] + 1$ đến i có thể được gom vào cùng một cuộc đối chiếu.

Mảng pre và cách tính: $pre[i]$ là chỉ số của "ngưỡng" cắt dưới i sao cho ta có thể "gộp" các nghi phạm từ $pre[i] + 1$ đến i vào cùng một cuộc đối chiếu hợp lệ.

Quá trình tính mảng pre được thực hiện theo dạng **sliding window** với hai chỉ số:

i: là chỉ số của nghi phạm hiện tại đang xét.

p: là chỉ số "cắt" bên trái, ban đầu $p = 0$.

Quy trình chi tiết như sau:

Lưu n khoảng $[li, ri]$ và vào mảng $seg[]$. Với mỗi nghi phạm i (từ 1 đến n):

Thêm khoảng của nghi phạm i vào "cửa sổ": (cộng 1 vào tất cả số từ $seg[i].l$ đến $seg[i].r$

Mục đích: "Thêm" phạm vi của nghi phạm i vào tập các khoảng đang xét để kiểm tra tính hợp lệ của cửa sổ: Sau khi thêm khoảng của nghi phạm i, ta lấy max (1 → MAX_R) trên cây để lấy giá trị "số lượng giao nhau" lớn nhất trên toàn bộ các chỉ số. Nếu giá trị này lớn hơn 1, có nghĩa là có ít nhất một điểm bị "bao phủ" bởi 2 hoặc nhiều khoảng – tức các nghi phạm trong cửa sổ hiện tại (từ p+1 đến i) không thể được gom vào cùng một cuộc đối chiếu hợp lệ.

Thu hẹp cửa sổ bằng cách loại bỏ các nghi phạm bên trái:
Trong vòng lặp **while (st.getmax(1, N) > 1),** ta thực hiện:
Tăng p lên: ++p: Loại bỏ khoảng của nghi phạm p khỏi cửa sổ:
→ trừ 1 cho các chỉ số từ $seg[p].L$ đến $seg[p].R$.

Quá trình này dừng lại khi cửa sổ từ $p + 1$ đến i **trở thành hợp lệ** (tức giá trị "max" trên cây phân đoạn không vượt quá 1). Sau khi cửa sổ hợp lệ được tìm thấy, gán pre[i] = p.

Ý nghĩa: Các nghi phạm từ p+1 đến i có thể được gom vào một cuộc đối chiếu vì không có sự giao nhau nào (vì mỗi điểm trên trục tọa độ chỉ bị bao phủ bởi tối đa 1 khoảng).

Code minh họa:

```
    for (int i = 1 -> n) {
        cin >> l, r;
        push(l, r) vào val;
        seg[i] = {l, r};
    }
```

Sắp xếp vào loại bỏ các cặp trùng lặp trong val

```
    int p = 0;
    for (int i = 1 -> n) {
        update(seg[i].L, seg[i].R, 1);
        while (getmax(1, N) > 1) { ++p; update(seg[p].L,
seg[p].R, -1);}
        pre[i] = p;
    }
```

Subtask 1, 4

Duyệt từ $Li \rightarrow Ri$, với công thức: $f[i] = f[pre[i]] + 1$.

Điều này có nghĩa là: số cuộc đối chiếu tối thiểu từ L đến i bằng số cuộc đối chiếu cần thiết từ L đến pre[i] cộng thêm 1 (vì ta "nhóm" các nghi phạm từ pre[i]+1 đến i vào một cuộc đối chiếu mới).

Kết quả: $f[R]$.

Độ phức tạp: $O(q * (R - L)) \sim O(q * n)$ cho trả lời truy vấn

Subtask 2, 3

Tuy n nhỏ ($n <= 5000$) nhưng $q <= 2e5$ nên $O(q * n) \sim 1e9$ → không khả thi Ta nhận thấy, với n nhỏ, ta có thể dùng bộ nhớ $O(n^2)$ để tiền xử lý bảng đáp án cho tất cả các khoảng $[i, j]$.

Cách làm tương tự subtask 1, 4

```
for (int i = 1->n) {
     for (int j = i->n) {
          ans[i][j] = ans[i][pre[j]] + 1;
     }
}
while (q--) {
     cin >> a, b;
     cout << ans[a][b];
}
```

Độ phức tạp: $O(n^2 + q)$

Subtask 5

Ta biết rằng dãy lineup tối ưu có thể được xây dựng bằng cách bắt đầu từ nghi phạm li và thêm các nghi phạm lần lượt từ trái sang phải vào dãy lineup hiện tại, cho đến khi không thể thêm được nữa. Khi gặp một nghi phạm không thể thêm vào, ta bắt đầu một dãy lineup mới và lặp lại quá trình này cho đến khi đạt đến nghi phạm ri. Và với thêm cách giải quyết subtask 1&4, ta sẽ suy nghĩ đến việc làm thế nào để "nhảy" nhanh đến những vị trí tối ưu nhất (vị trí "không thể thêm được nữa") mà không cần duyệt như các subtask trên.

Ta có thể dùng *binary lifting* để làm việc này.

Ở đây do nhảy từ $li \rightarrow x1 \rightarrow \ldots \rightarrow x \rightarrow ri$ nên thay vì dùng mảng $pre[]$ ta sẽ dùng mảng $suf[]$ với ý nghĩa tương tự $pre[]$ nhưng khác là lần này ở vị trí bên trái và là vị trí xa nhất có thể (tức là từ $i \rightarrow suf[i]$ chứ không còn là từ $pre[i] + 1 \rightarrow i$).

Cách xây dựng $suf[]$ tương tự $pre[]$ (nhưng ở đoạn code ta sẽ dùng set cho đỡ dài dòng, không dùng segment tree).

Lúc tính xong mảng $suf[]$, ta chỉ cần chạy thuật toán *binary lifting* giải quyết từng truy vấn trong $log(n)$

2.3.3.3 Đánh giá

- Thuật toán sử dụng kĩ thuật *Binary Lifting* để trả lời các truy vấn một cách hiệu quả với độ phức tạp $O(logN)$, đồng thời sử dụng *set* để quản lý các đoạn một cách hợp lý với độ phức tạp $O(NlogN)$
- Tổng độ phức tạp cho N đoạn và Q truy vấn sẽ là $O(NlogN + QlogN)$.

2.3.4 Bài 4 – MINID

2.3.4.1 Đề bài

https://codeforces.com/contest/588/problem/E

Một quốc gia có n thành phố (được đánh số từ 1 đến n) và $n - 1$ con đường hai chiều. Mỗi con đường kết nối hai thành phố khác nhau. Có một con đường duy nhất giữa bất kỳ hai thành phố nào.

Có m người sống ở quốc gia đó, được đánh số từ 1 đến m. Mỗi người có một số ID. là số thứ tự của họ; người thứ i sống ở thành phố số c_i. Lưu ý là có thể có nhiều người sống ở cùng một thành phố, và cũng có thể có thành phố không có ai ở cả.

Mỗi truy vấn cho các số v, u và a:

- Giả sử có x người sống ở các thành phố nằm trên con đường từ thành phố v đến thành phố u. ID của những người này là $p_1, p_2, \ldots, p_x$ theo thứ tự tăng dần.
- Nếu $k = min(x, a)$, thì cần tìm k ID nhỏ nhất trong những người đó. Nói cách khác ta cần tìm tối đa a ID nhỏ nhất trong các thành phố từ v đến u

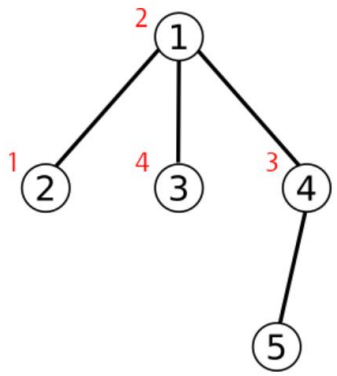
Dữ liệu vào: Cho trong file **MINID.INP**

- Dòng đầu tiên chứa ba số nguyên n, m và q ($1 \leq n, m, q \leq 10^5$).
- $n - 1$ dòng tiếp theo, mỗi dòng chứa hai số nguyên v và u – một con đường nối 2 thành phố ($1 \leq v, u \leq n, v \neq u$).
- Dòng tiếp theo chứa m số nguyên $c_1, c_2, \ldots, c_m$ ($1 \leq c_i \leq n$; $1 \leq i \leq m$).
- q dòng tiếp theo, mỗi dòng chứa ba số nguyên v, u và a ($1 \leq v, u \leq n$ và $1 \leq a \leq 10$).

Kết quả ra: In ra file **MINID.OUT**

- Đối với mỗi truy vấn, in các số $k, p_1, p_2, \dots, p_k$ được phân tách bằng khoảng trắng trong một dòng. Trong đó, k là số lượng ID nhỏ nhất và $p_1, p_2, \dots, p_k$ là các ID theo thứ tự tăng dần.

Ví dụ:

MINID.INP	MINID.OUT	Minh họa
5 4 5 1 3 1 2 1 4 4 5 2 1 4 3 4 5 6 1 5 2 5 5 10 2 3 3 5 3 1	1 3 2 2 3 0 3 1 2 4 1 2	

2.3.4.2 Thuật toán

Để tìm người sống trên đoạn đường đi từ thành phố v đến thành phố u thực hiện các bước:

Tính $LCA(v, u)$ bằng kĩ thuật $Binary\ Lifting$.

Để lấy danh sách người sống trên đoạn đường đi này, ta hợp nhất các danh sách người sống tại các thành phố trên đường đi từ v đến LCA, từ u đến LCA và tại LCA. Danh sách người sống tại mỗi thành phố được lưu trong mảng **save.** Sử dụng hàm Merge để hợp nhất các danh sách:

Hợp nhất 2 danh sách theo thứ tự tăng dần

Sau khi hợp nhất, loại bỏ các ID trùng lặp và giữ lại tối đa 10 ID nhỏ nhất.

Với mỗi truy vấn, ta cần trả về tối đa a người có ID nhỏ nhất từ danh sách đã hợp nhất. Nếu có ít người hơn a trả về tất cả.

2.3.4.3 Đánh giá

Thuật toán sử dụng DFS để duyệt cây và xây dựng các thông tin cần thiết về các tổ tiên của mỗi thành phố với Binary Lifting, giúp trả lời các truy vấn về đường đi trong cây trong thời gian $O(log\ n)$ cho mỗi truy vấn. Đồng thời, thuật toán sử dụng kĩ thuật hợp nhất danh sách để xử lý và trả về các người có ID nhỏ nhất trên đường đi từ v đến u.

Độ phức tạp cho phần DFS và chuẩn bị Binary Lifting: $O(n\ log\ n)$.

Độ phức tạp cho mỗi truy vấn: $O(log\ n)$ cho việc tìm LCA và $O(1)$ cho việc hợp nhất các danh sách.

Thời gian cho q truy vấn: $O(qlog\ n)$.

Tổng độ phức tạp của thuật toán là $O(nlogn + qlogn)$.

2.3.5 Bài 5 – Double Tree

2.3.5.1 Đề bài

https://codeforces.com/problemset/problem/1140/G

Bạn được cho một đồ thị vô hướng đặc biệt. Đồ thị này gồm $2 \times n$ đỉnh được đánh số từ 1 đến $2 \times n$. Đồ thị có các đặc điểm sau:

- Đồ thị có chính xác $3n - 2$ cạnh: n cạnh kết nối các đỉnh có số lẻ với các đỉnh có số chẵn, $n - 1$ cạnh kết nối các đỉnh có số lẻ với nhau, và $n - 1$ cạnh kết nối các đỉnh có số chẵn với nhau;
- Đối với mỗi cạnh (u, v) giữa một cặp đỉnh có số lẻ, tồn tại một cạnh $(u + 1, v + 1)$, và ngược lại;
- Đối với mỗi số lẻ $u \in [1,2n - 1]$, tồn tại một cạnh $(u, u + 1)$;
- Đồ thị là liên thông; hơn nữa, nếu ta xóa tất cả các đỉnh có số chẵn khỏi đồ thị, và tất cả các cạnh nối với chúng, đồ thị sẽ trở thành một cây (điều này cũng áp dụng khi xóa các đỉnh số lẻ).

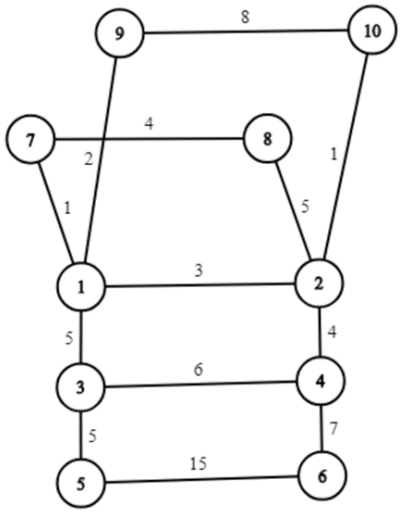
Vì vậy, đồ thị có thể được biểu diễn dưới dạng hai cây có cấu trúc giống nhau, và n cạnh nối mỗi đỉnh của cây thứ nhất với đỉnh tương ứng của cây thứ hai.

Các cạnh của đồ thị đều có trọng số. Độ dài của một đoạn đường đơn giản trong đồ thị là tổng trọng số của các cạnh đã đi qua.

Bạn được cho q truy vấn với đồ thị này; trong mỗi truy vấn, bạn cần tính toán độ dài của đoạn đường ngắn nhất giữa một cặp đỉnh trong đồ thị. Bạn có thể trả lời tất cả các truy vấn không?

- **Dữ liệu vào:** Cho trong file **DOUBLETREE.INP**
  - Dòng đầu chứa số nguyên n $2 \leq n \leq 3 \times 10^5$
  - Dòng thứ 2 chứa n số nguyên $w_{1,2}, w_{3,4}, \dots, w_{2n-1,2n}$ $(1 \leq w_{i,i+1} \leq 10^{12})$ là các số nguyên mô tả trọng số của các cạnh kết nối các đỉnh số lẻ với các đỉnh số chẵn
  - $n - 1$ dòng tiếp theo, dòng thứ i chứa 4 số nguyên $x_i, y_i, w_{i,1}, w_{i,2}$ $(1 \leq x_i, y_i \leq n, x_i \neq y_i, 1 \leq w_{i,j} \leq 10^{12})$ mô tả 2 cạnh: 1 cạnh nối $2x_i - 1$ và $2y_i - 1$ có trọng số $w_{i,1}$; 1 cạnh nối $2x_i$ và $2y_i$ có trọng số $w_{i,2}$
  - Dòng tiếp theo chứa 1 số nguyên q $(1 \leq q \leq 6 \times 10^5)$ là số lượng truy vấn.
  - q dòng tiếp theo, dòng thứ i chứa 2 số nguyên u_i và v_i $(1 \leq u_i, v_i \leq 2n, u_i \neq v_i)$ mô tả 1 truy vấn "tính độ dài đoạn đường ngắn nhất giữa các đỉnh u_i và v_i ".
- **Kết quả ra:** Ghi vào file **DOUBLETREE.OUT**
  - In ra q số nguyên, số nguyên thứ i là câu trả lời cho truy vấn i

- **Ví dụ:**

DOUBLETREE.INP	DOUBLETREE.OUT	Hình minh họa
5 3 6 15 4 8 1 2 5 4 2 3 5 7 1 4 1 5 1 5 2 1 3 1 2 5 6 1 10	3 15 4	

2.3.5.2 Thuật toán

Trước tiên, ta cần tìm đường đi ngắn nhất giữa các cặp nút dạng $2i - 1$ và $2i$. Việc này có thể thực hiện bằng quy hoạch động trên cây.

Lưu ý rằng bất kỳ đường đi ngắn nhất nào như vậy sẽ chỉ đi qua đúng một cạnh nối giữa cây chẵn và cây lẻ.

Việc thay thế các cạnh "giao nhau" này bằng một khoảng cách cố định sẽ không làm thay đổi đường đi ngắn nhất, nhưng sẽ giúp việc xử lý bài toán dễ hơn.

Tiếp theo, để tìm đường đi ngắn nhất, ta có 2 cách:

- Sử dụng thuật toán phân tách trọng tâm (centroid decomposition), ta có thể tìm đường đi ngắn nhất từ gốc đến một nút bằng cách:

* Định nghĩa $dp[i][j]$ là độ dài đường đi ngắn nhất từ nút $2i - j$ đến gốc, với một mảng DP khác cho cây còn lại.

- Sử dụng $binary\ lifting$, ta có thể định nghĩa:

* $dp[i][j][k][l]$ là khoảng cách từ nút $2i - j$ đến tổ tiên bậc 2^l thuộc một trong hai cây (được theo dõi bằng chỉ số k, biểu thị là đang ở cây chẵn hay cây lẻ).

Dù theo cách nào: nếu ta muốn tìm đường đi từ một nút đến nút khác, ta phải đi qua đường đơn trên cây giữa hai nút đó, có thể có sự "nhảy qua lại" giữa hai cây (chẵn và lẻ).

2.3.5.3 Đánh giá

Chương trình này thực hiện các phép toán trên đồ thị với các thuật toán tối ưu hóa, sử dụng phương pháp *Binary Lifting* cho bài toán Lowest Common Ancestor (LCA) và thuật toán Dijkstra để tìm đường đi ngắn nhất trong đồ thị. Tính toán các quãng đường từ các đỉnh tới LCA

- Thời gian tiền xử lý (Dijkstra + DFS + Binary Lifting): $O((n + m) \log n)$, trong đó n là số đỉnh và m là số cạnh.
- Thời gian cho mỗi truy vấn: $O(\log n)$.
- Tổng độ phức tạp cho tất cả các truy vấn: $O(q \log n)$, trong đó q là số lượng truy vấn.

2.3.6 Một số bài tập khác

1) https://codeforces.com/gym/102694/problem/C
2) https://cses.fi/problemset/task/1750/
3) https://cses.fi/problemset/task/1160
4) https://cses.fi/problemset/task/1191/
5) https://zibada.guru/gcj/ks2021h/problems/#D

3. PHẦN KẾT LUẬN

Trong chuyên đề này, chúng ta đã đi qua các khái niệm cơ bản và nâng cao về kĩ thuật *Binary Lifting*, từ lý thuyết cho đến ứng dụng trong giải quyết các bài toán thuật toán phức tạp. *Binary Lifting* là một kĩ thuật tối ưu hóa mạnh mẽ, đặc biệt hữu ích trong các bài toán yêu cầu truy vấn cây nhị phân hoặc các bài toán tìm kiếm cha của các đỉnh trong các cấu trúc dữ liệu phức tạp như cây, đồ thị.

Qua các bài tập thực hành, tôi đã trình bày được các cơ hội áp dụng kĩ thuật này vào việc tính toán các truy vấn trên cây, đồ thị, đến các bài toán tìm kiếm cha, xử lý các bài toán liên quan đến đường đi ngắn nhất hoặc bài toán duyệt cây. Những bài tập này không chỉ giúp củng cố lý thuyết mà còn tăng cường kỹ năng lập trình và khả năng giải quyết bài toán.

Binary Lifting không chỉ là một công cụ hữu ích trong các cuộc thi lập trình mà còn có thể ứng dụng trong nhiều bài toán thực tế, từ việc tối ưu hóa truy vấn dữ liệu đến các bài toán trong lĩnh vực khoa học máy tính như hệ thống cơ sở dữ liệu, đồ họa máy tính, và trí tuệ nhân tạo.

4. TÀI LIỆU THAM KHẢO

1. https://www.geeksforgeeks.org
2. https://wiki.vnoi.info/algo/data-structures/lca-binlift.md
3. https://cp-algorithms.com/graph/lca_binary_lifting.html
4. https://www.geeksforgeeks.org/binary-lifting-guide-for-competitive-programming/
5. https://usaco.guide/plat/binary-jump#problems
6. https://leetcode.com/discuss/post/4139771/binary-lifting-by-abhavgoel-cel6/
7. https://codeforces.com/blog/entry/61364
8. https://codeforces.com/blog/entry/100826
9. https://aryansh.gitbook.io/informatics-notes/binary-lifting/binary-lifting-gold-intro